Inventive Media 👋

Getting Started With CSS

Introduction to CSS

Day 2

KEY TOPICS

1. **What is CSS**

We will discuss about CSS definition of terms, Purpose and How CSS Works.

CSS Origin

2. **Types of CSS Styling**

What are the differences between Inline, Internal and External styling in CSS.

CSS Techniques

3. **The CSS Syntax**

We will discuss all about CSS syntax from selectors, box model, font, colors, background, animations and more.

CSS hands on!

What is CSS?

What is CSS?

CSS stands for Cascading Style Sheets

It is used to control the appearance and layout of HTML elements on a webpage. CSS controls **how** HTML elements **look** on a webpage (layout, colors, fonts, etc.).

Think of CSS as the “clothing” that styles the “body” of a webpage (HTML).

Why use CSS?

✅ **Separates Content from Design**
HTML handles the content, while CSS handles the look.

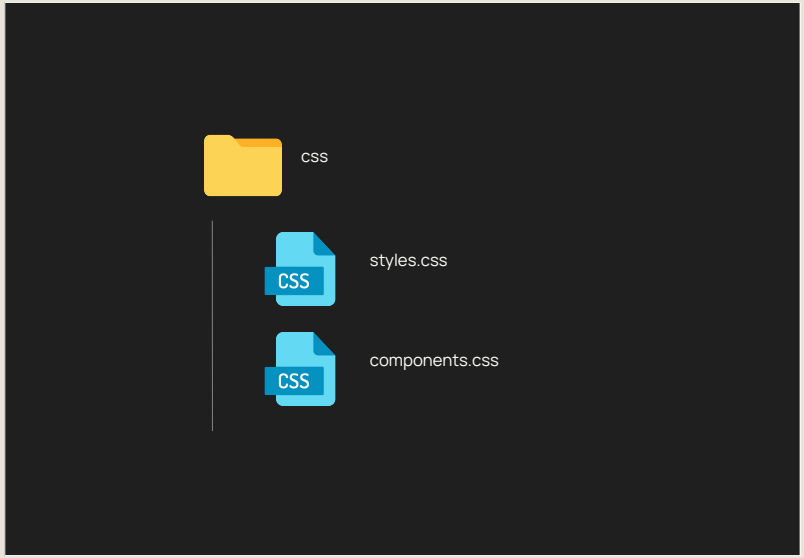
✅ **Saves Time**
You can style multiple pages with just one CSS file.

✅ **Improves Consistency**
Makes your website look clean and professional.

✅ **Responsive Design**
Easily adapt styles for mobile, tablet, and desktop.

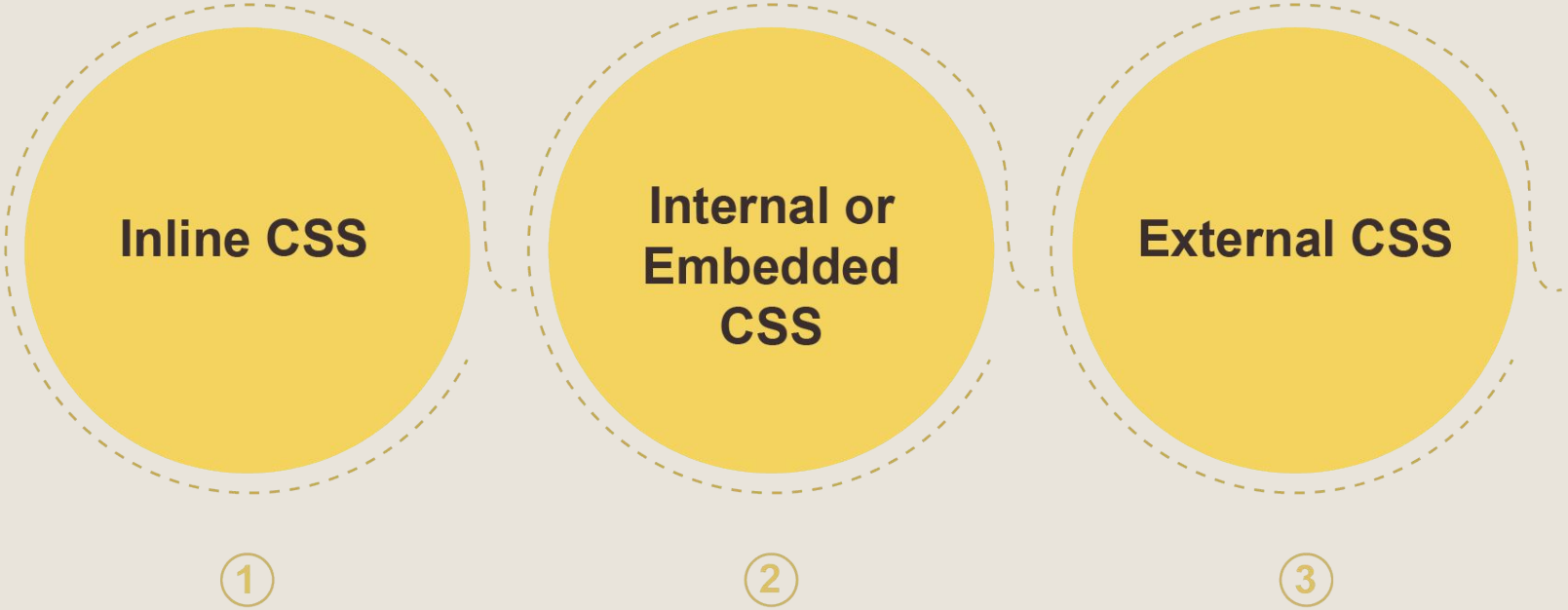
CSS File Extension

- File extension: `.css`
- You can use **multiple CSS files** in large projects
- Essential for **styling web apps**



Types of CSS Styling

Types of CSS



Types of CSS Styling

Inline Style (CSS)

- Harder to maintain because styles are written directly inside HTML elements. You have to edit each element one by one if you want to make a change.
- Not reusable. You must repeat styles for every element, which leads to more lines of code.
- Breaks the principle of separating content (HTML) from presentation (CSS), which can make your code messy.

```
<p style="color: red;">Hello</p>
```

Types of CSS Styling

Internal Style (CSS)

- Easier than inline – styles are placed inside a `<style>` tag in the `<head>` of the HTML file. Still not ideal for large projects.
- Reusable within the same page only. Not ideal for multi-page websites.
- Partially separates content and style, but still in the same file. Better than inline, but still cluttered.

```
<head>
  <title>Document</title>
  <style>
    p {
      color: red;
    }
  </style>
</head>
<body>
  <p>Hello</p>
</body>
</html>
```

Types of CSS Styling

External Style (CSS)

- Most maintainable – all styles are stored in a separate .css file. You can update styles for multiple pages from one place.
- Highly reusable across multiple pages and projects by linking the same stylesheet.
- Follows best practice – keeps HTML clean and separates layout from content. Great for teamwork and version control.

HTML

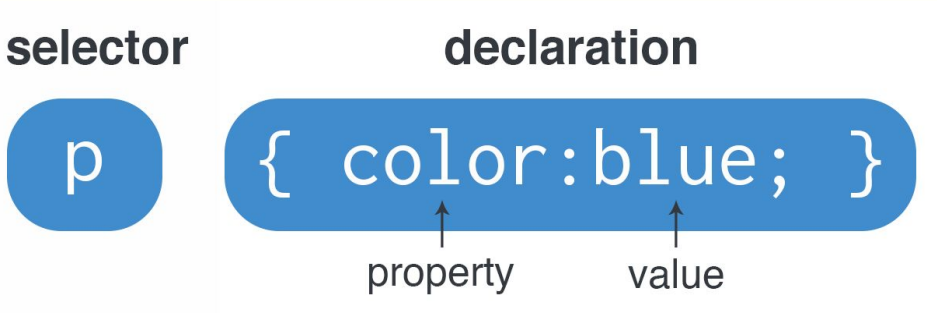
```html
<html lang="en">
<head>
  <title>Document</title>
  <link rel="stylesheet" href="./styles.css">
</head>
<body>
  <p>Hello</p>
</body>
</html>
```

CSS

```css
p {
   color: red;
}
```

The CSS Syntax

Single Line Style



Block-Level Style (Recommended)

CSS Comments

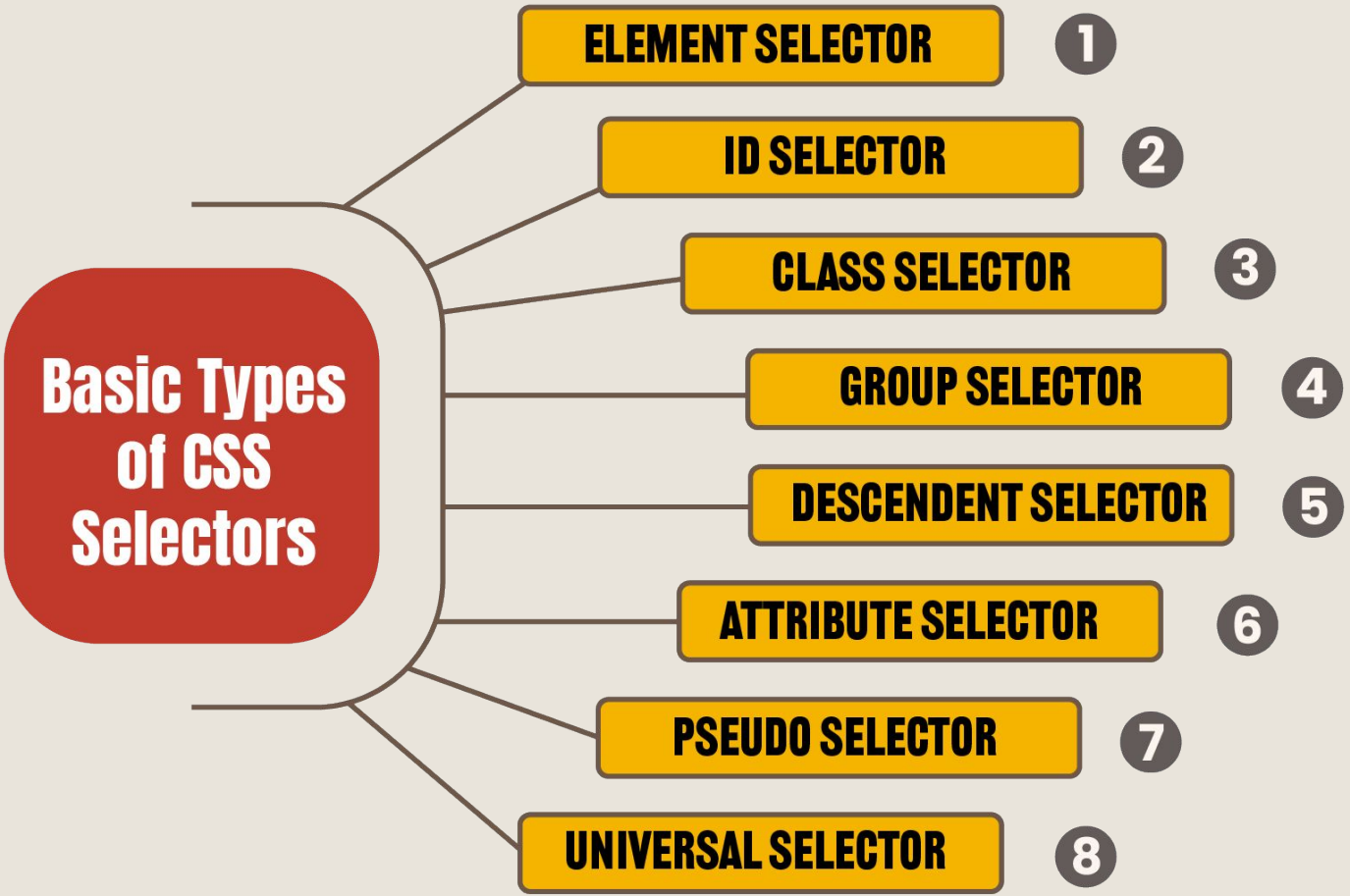
CSS Basics

CSS Comments

- used to add explanatory notes to the code or to prevent the browser from interpreting specific parts of the style sheet.

```css
/* This is a comment */
/*
  This is
  multi line
  comment
*/
```

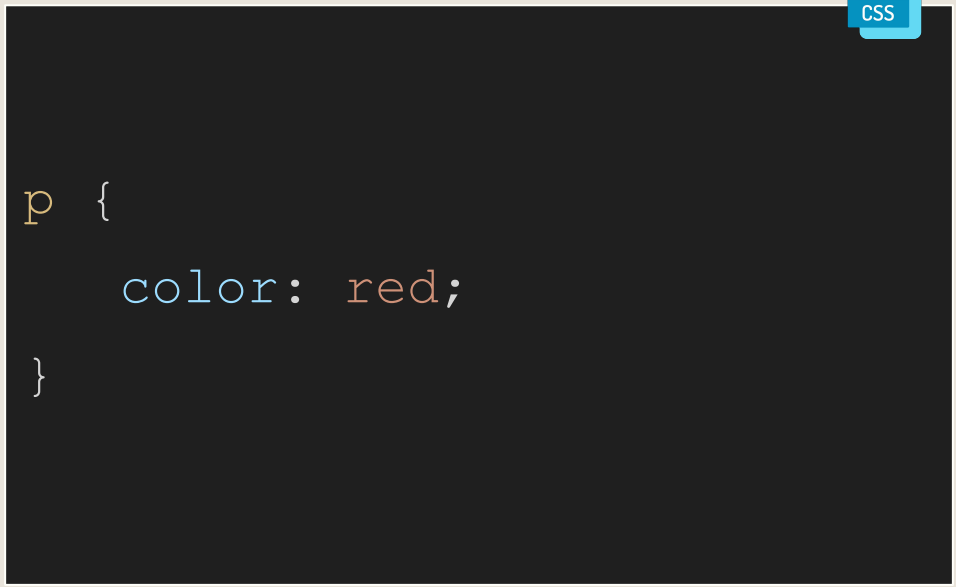
CSS Selectors


Basic Types of CSS Selectors
ELEMENT SELECTOR
1
ID SELECTOR
2
CLASS SELECTOR
3
GROUP SELECTOR
4
DESCENDENT SELECTOR
5
ATTRIBUTE SELECTOR
6
PSEUDO SELECTOR
7
UNIVERSAL SELECTOR
8

CSS Selectors

Element Selector

- Targets an element directly by name whether its h1, div, p or any html element.



Some Common Element	Description
`h1` to `h6`	Headings (from largest to smallest)
`p`	Paragraph
`div`	Generic container (block-level)
`span`	Inline container
`a`	Anchor (hyperlink)
`img`	Image
`ul`, `ol`, `li`	Lists (unordered, ordered, list items)
`table`, `tr`, `td`, `th`	Table and its parts
`input`, `textarea`, `button`, `select`, `label`	Form elements
`nav`, `header`, `footer`, `section`, `article`	Semantic elements

CSS Selectors

Class Selector

- Targets an element via class name
- Class names are attribute that can be attached to an html tag
- Starts with (.) indicating that you are targeting a class

HTML

```
<p class="text">Hello</p>
```

CSS

```
.text {
    color: red;
}
```

CSS Selectors

id Selector

- Targets an element via id
- id's are attribute that can be attached to an html tag
- Starts with # as indicator that you are targeting an id

HTML

```html
<p id="greet">Hello</p>
```

CSS

```css
#greet {
    color: red;
}
```

CSS Selectors

Group selector

- Target multiple elements, or class or id
- Apply the same styles with selected elements

HTML

```
<p class="text">Hello</p>
<p id="greet">I'm John Doe</p>
<p>I'm 30 years old</p>
```

CSS

```
.text, #greet, a {
    color: red;
}
```

CSS Selectors

Descendant selector

- Target an element inside other element
- Can be a combination of element, class or id

HTML

```
<div class="box-greet">
    <p>I'm 30 years old</p>
</div>
```

CSS

```
.box-greet p {
   color: red;
}
```

CSS Selectors

Parent > Direct Child selector

- The child selector selects elements that are direct children only of a specified parent.

```html
<ul>
  <li>Blue item</li> <!-- Direct child -->
  <div>
    <li>Not blue (not a direct child)</li>
<!-- Nested deeper -->
  </div>
</ul>
```

```css
ul > li {
  color: blue;
}
```

Attribute Selector

CSS attribute selectors allow you to style HTML elements based on the presence or value of their attributes.

Why Use Them?

- Target specific elements precisely
- Avoid adding extra classes or IDs
- Useful for forms, links, and dynamic content

CSS

```css
/* CSS */
input[required] {
  border: 2px solid red;
}
```

HTML

```html
<!-- HTML -->
<input type="text" required>
<input type="text">
```

Pseudo-Classes Selector

A pseudo-class selector is used to define a special state, position, or condition of an HTML element, without needing to add a class or ID to the element.

Types of Pseudo-Classes

- State Pseudo-Class
- Condition Pseudo-Class

State Pseudo-Class

```
a:link {
  color: blue;
}
a:visited {
  color: red;
}
a:hover {
  color: green;
}
a:active {
  color: yellow;
}
```

Condition Pseudo-Class

:first-child

```html
<!DOCTYPE html>
<html>
<head>
  <style>
    li:first-child {
      color: red;
    }
  </style>
</head>
<body>
  <h3>:first-child Example</h3>
  <ul>
    <li>First item (Red)</li>
    <li>Second item</li>
    <li>Third item</li>
  </ul>
</body>
</html>
```

:first-child Example

- First item (Red)
- Second item
- Third item

:last-child

```html
<!DOCTYPE html>
<html>
<head>
  <style>
    li:last-child {
      color: blue;
    }
  </style>
</head>
<body>
  <h3>:last-child Example</h3>
  <ul>
    <li>First item</li>
    <li>Second item</li>
    <li>Last item (Blue)</li>
  </ul>
</body>
</html>
```

:last-child Example

- First item
- Second item
- Last item (Blue)

:nth-child

```html
<!DOCTYPE html>
<html>
<head>
  <style>
    li:nth-child(2) {
      color: green;
    }
    li:nth-child(odd) {
      background-color: #f0f0f0;
    }
  </style>
</head>
<body>
  <h3>:nth-child Example</h3>
  <ul>
    <li>Item 1 (odd)</li>
    <li>Item 2 (Green)</li>
    <li>Item 3 (odd)</li>
    <li>Item 4</li>
  </ul>
</body>
</html>
```

:nth-child Example

- Item 1 (odd)
- Item 2 (Green)
- Item 3 (odd)
- Item 4

Pseudo-Class	Description
`:link`	Targets unvisited hyperlinks
`:hover`	When the mouse hovers over the element
`:active`	When the element is being clicked
`:visited`	Targets visited hyperlinks
`:focus`	When an input field is focused
`:disabled`	When a form element is disabled
`:checked`	When a checkbox or radio is selected
`:first-child`	Targets the first child in a parent
`:last-child`	Targets the last child in a parent
`:nth-child(n)`	Targets the nth child in a parent

Pseudo-Elements Selector

A pseudo-element allows you to style specific parts of an element — like the first letter or inserting generated content.

Pseudo-Elements	Description
`::before`	Inserts content before an element
`::after`	Inserts content after an element
`::first-letter`	When the element is being clicked
`::first-line`	Targets visited hyperlinks
`::selection`	When an input field is focused

CSS Hierarchy

When multiple CSS rules target the same element, CSS hierarchy decides which rule is applied.

1. Specificity

Selector Type	Specificity Value
Inline styles	`1000`
ID selector `#id`	`0100`
Class `.class`, Pseudo-class `:hover`	`0010`
Element `div`, `p`	`0001`

2. Source Order

```
p {
  color: blue;  /* gets overridden */
}

p {
  color: red;   /* this one is applied */
}
```

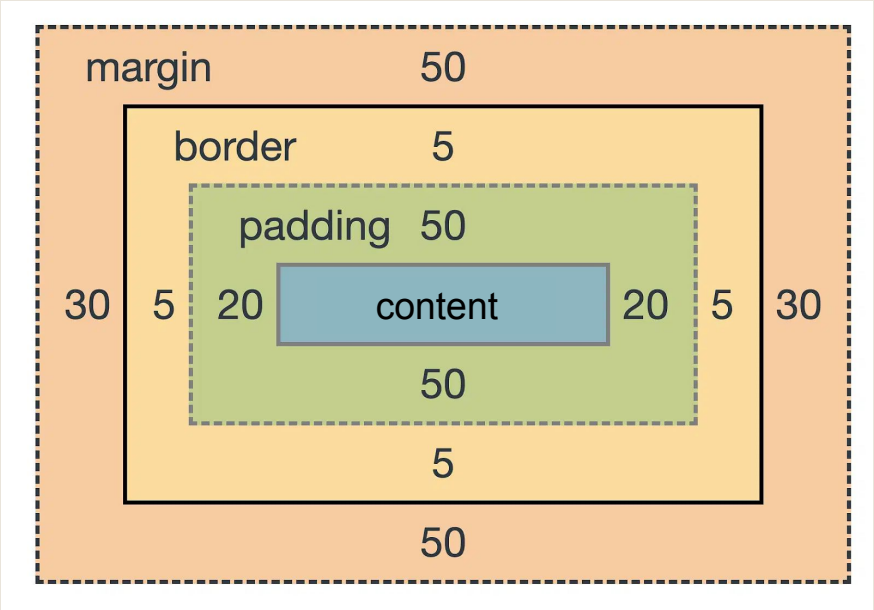
2. !important

`!important` → overrides all (except another `!important` with higher specificity)

Box Model

CSS Box Model

Every HTML element is treated as a rectangular box by the browser, The CSS Box Model describes how these boxes are structured, sized, and spaced.



- Content - the actual text, image, or media inside the element.
- Padding - space between the content and the border.
- Border - a visible or invisible line around the padding.
- Margin - space outside the border.

Box-sizing: Controls if width and height includes padding and border

Margin VS Padding

Longhand code

```
.text-margin {
    margin-top: 40px;
    margin-left: 40px;
    margin-bottom: 40px;
    margin-right: 40px;
}
```

```
.text-padding {
    padding-top: 40px;
    padding-left: 40px;
    padding-bottom: 40px;
    padding-right: 40px;
}
```

Margin VS Padding

Shorthand code

All Four Sides

1 value ———————— margin / padding: 10px;

Top & Bottom Left & Right

2 values ———————— margin / padding: 10px 20px;

Top Left & Right Bottom

3 values ———————— margin / padding: 10px 20px 30px;

Top Right Bottom Left

4 values ———————— margin / padding: 10px 20px 30px 40px

Border

Border Thickness

- px
- em
- rem

```
border-width: 5px;
```

Border Line Style

- solid
- dashed
- dotted
- double
- groove
- ridge
- insert
- outset

```
border-style: dashed;
```

Border Color

- rgb
- hex

```
border-color: #ff0000;
```

Border

Longhand code

```
.box {
    border-top: 4px;
    border-left: 4px;
    border-bottom: 4px;
    border-right: 4px;
    border-style: solid;
    border-color: blue;
}
```

Border

Shorthand code

```
.box {
    border: 4px dashed #bdba22;
}
```

border-width | border-style | border-color

Border Radius

makes the corners of a box rounded instead of sharp.

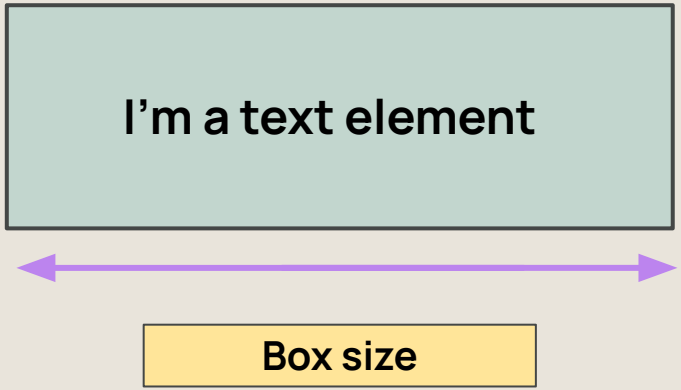
```
border-radius: 40px;
```

```
border-radius: 10px 20px;
```

```
border-top-left-radius: 10px;
border-top-right-radius: 10px;
border-bottom-right-radius: 10px;
border-bottom-left-radius: 10px;
```

Box-sizing

- **box-sizing** controls how the total size of an element is calculated
- It helps you predict how big a box will really appear on the page.



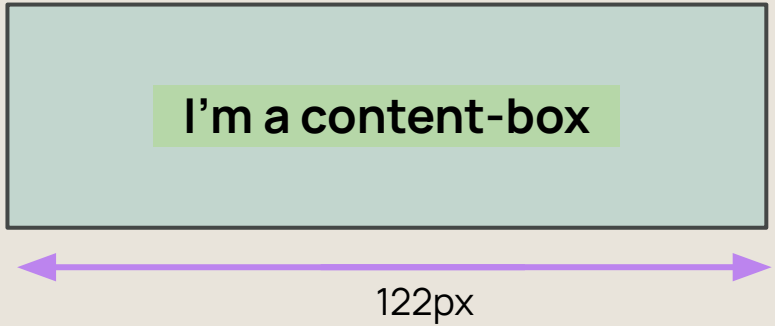
Box-sizing

Value	Description
content-box	Width & height only include the content. Padding & border are added outside, making the box bigger
border-box	Width & height include content + padding + border together. The box stays the same size on the page.

```
.box {
    box-sizing: content-box;
}
```

```
.box {
    box-sizing: border-box;
}
```

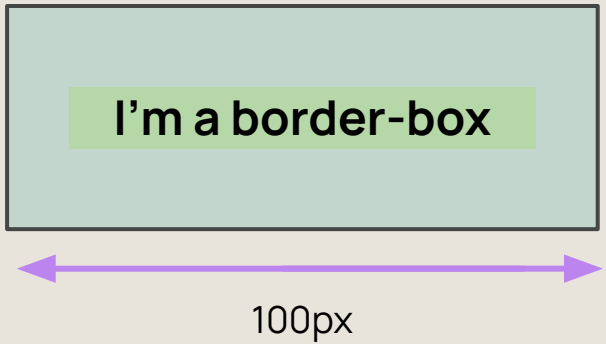
Content-box VS Border-box



Total width = border-left + padding-left + width + right-padding + border-right

Total width = 1px + 10px + 100px + 10px + 1px

Total width = 122px



Total width = 100px

```
.box {
    width: 100px;
    padding: 10px;
    border: 1px solid black;
}
```

WIDTH AND HEIGHT ELEMENT CAN BE SET ON SOME INLINE ELEMENTS BUT NOT OTHERS

CAN

- img
- input
- select
- textarea
- button

CAN'T

- span
- b
- i
- strong
- em
- label
- a

Using Display

Feature / Behavior	`inline`	`inline-block`	`block`
Starts on a new line?	❌ No	❌ No	✅ Yes
Width/Height can be set?	❌ No	✅ Yes	✅ Yes
Respects padding & margin?	❌ Partially (horizontal only)	✅ Yes	✅ Yes
Takes full width by default?	❌ No	❌ No	✅ Yes
Stacking behavior	Appears beside other elements	Appears beside other elements	Pushes other content below
Common examples	`<span>`, `<a>`, `<strong>`	`<img>`, `<button>`, `<input>` (custom styled)	`<div>`, `<p>`, `<section>`, `<form>`
Use case	For text-level styling	For layout with sizing **without** full line	For larger sections or layout blocks

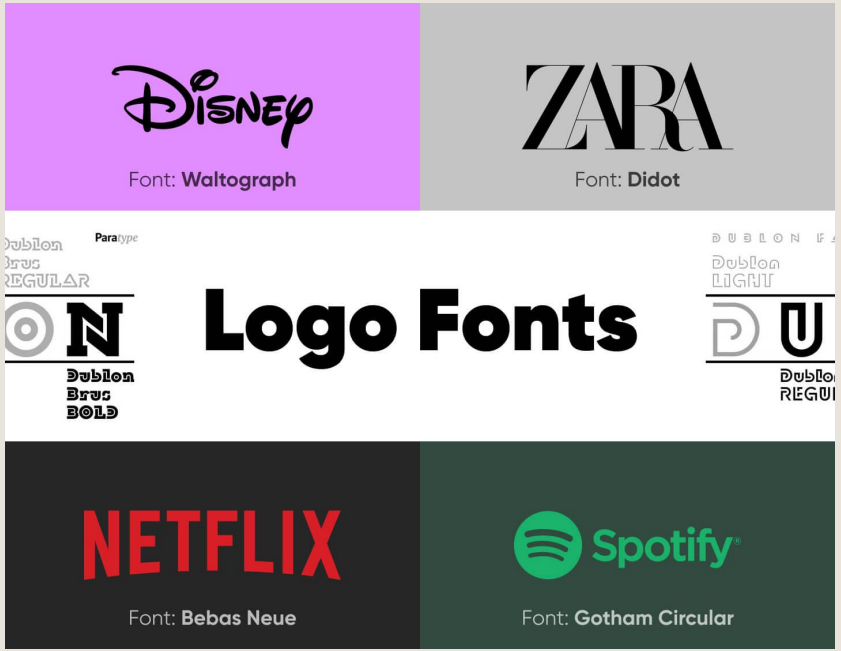
Width & Height Properties

Property	Description	Works on `block`?	Works on `inline`?	Notes / Behavior
`width`	Sets the exact width of the element	✅ Yes	❌ No (ignored)	Use `inline-block` if sizing needed
`max-width`	Sets the maximum width an element can grow to	✅ Yes	❌ No	Helpful for responsive layouts
`min-width`	Sets the minimum width an element can shrink to	✅ Yes	❌ No	Ensures content doesn't get too narrow
`height`	Sets the exact height of the element	✅ Yes	❌ No (ignored)	Needs display `block` or `inline-block`
`max-height`	Limits the maximum height	✅ Yes	❌ No	Often used for scrollable boxes
`min-height`	Ensures a minimum height	✅ Yes	❌ No	Keeps layout from collapsing

Font styling

Font styles

a property used to specify the style of a font, primarily for displaying text in either a normal, italic, or oblique style. It is one of several properties within the font shorthand property that control the appearance of text.



Font styles

Property	Description
color	Changes the text color
font-size	Sets the size of the text
font-family	Specifies the typeface (ex Arial, Sans-serif)
font-weight	Sets thickness of the font: normal, bold, etc
text-transform	Changes text case: uppercase, lowercase, capitalize
line-height	Sets the vertical spacing between lines of text
letter-spacing	Adjust space between letters

Colors in CSS

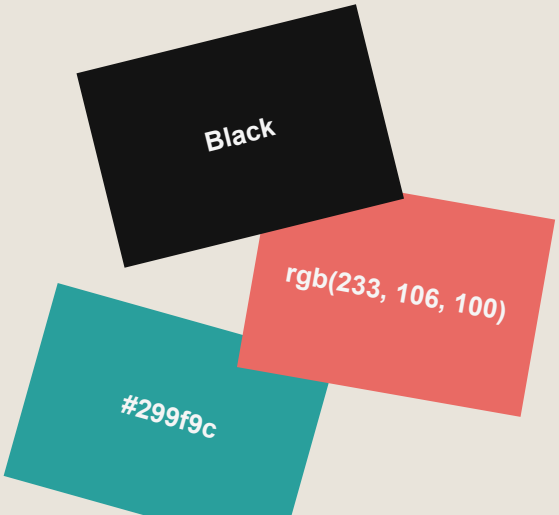
CSS Colors

Color in CSS refers to how we visually represent text, backgrounds, borders, and other elements on a web page using specific **color values**.

Colors in CSS are not just about beauty — they improve **usability**, **readability**, and **accessibility**.

They play a huge role in:

- Drawing attention to important parts
- Conveying mood or branding
- Creating contrast between elements



CSS Colors

Format	Example	Supports Transparency	Commonly Used?	Best For
HEX	`#FF5733`	❌ (except `#RRGGBBAA` in modern browsers)	✅ **Very Common**	Fast, compact code, static colors
RGB	`rgb(255, 87, 51)`	❌	✅ Common	JavaScript color manipulation, precise values
RGBA	`rgba(255, 87, 51, 0.5)`	✅	✅ Common	Semi-transparent colors (overlays, buttons, hover)
HSL	`hsl(14, 100%, 60%)`	❌ (`hsla` supports transparency)	⬛ Growing in use	Theming, easier color adjustment (light/dark)
HSLA	`hsla(14, 100%, 60%, 0.5)`	✅	⬛ Sometimes used	Themed transparency, color manipulation

CSS Colors

How to Set Color?

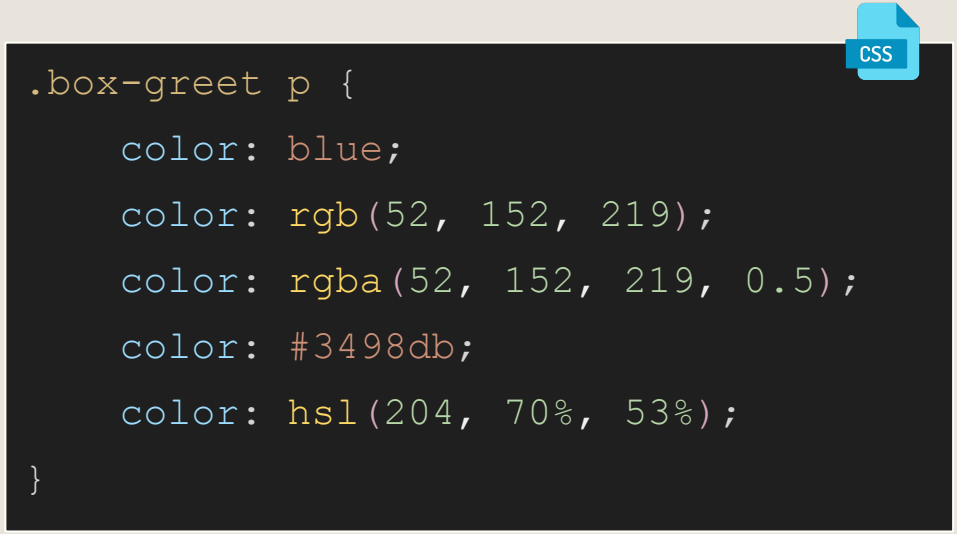
- Use the color property for text
- Use the background-color property for backgrounds

```
p {
    color : red;
    background-color : yellow;
}
```

CSS Colors

Ways to Write Colors

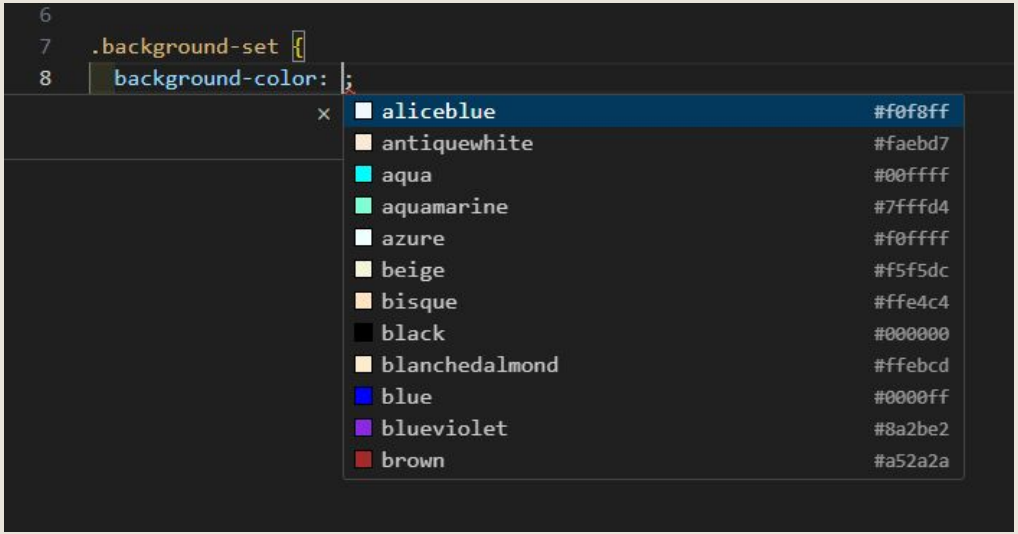
- Named Colors
- RGB
- RGBA
- Hex Codes
- HSL



Named colors

The set of colors VS Code shows is based on the **standard CSS color keywords** plus its built-in **color picker tool**, all part of **IntelliSense** and **language support for CSS**

A predefined list of 147 color keywords from CSS3



Named colors

- Use predefined names like red, green, blue, etc.
- Browsers only recognise 147 named colors

```css
.box-greet p {
    color: red;
    color: blue;
    color: green;
}
```

red

aqua

green

fuchsia

blue

slategray

⚠️ Tip: Named colors are fine for practice, but use HEX, RGB, or HSL for real projects to match exact brand colors.

Hex code

Hex code (short for **hexadecimal code**) is a way to represent **colors** in CSS using a combination of **numbers and letters**.

It starts with a # symbol, followed by **6 characters** (or 3 in shorthand), and tells the browser **how much red, green, and blue (RGB)** to use for a color.

```
.background-set {
  background-color: #ff0000;
}
```

Hex code

- great for picking exact colors in design, but you can't easily tell what color it is just by reading the code.

```css
.box-greet p {
    color: #3498db;
}
```

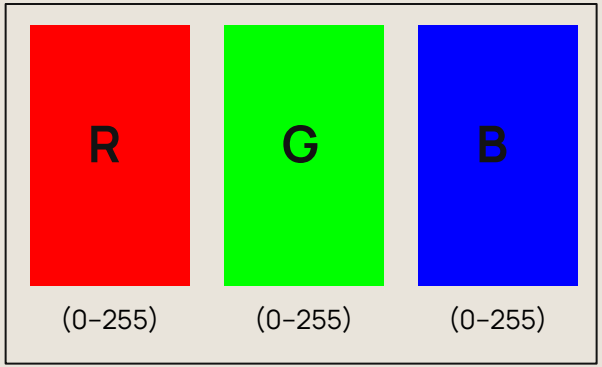

= #AAFF00

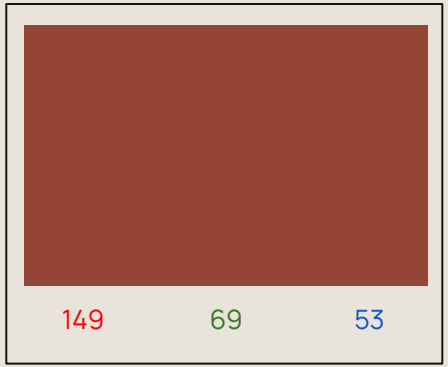
The first two characters are for red, the next two for green, and the last two for blue — each pair goes from 00 (none) to ff (full).

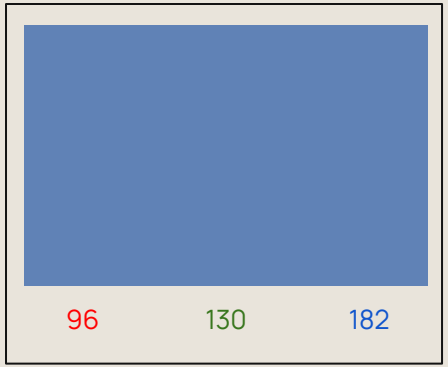
RGB

- uses numbers for Red, Green, and Blue (0–255).
- Good for precise colors, but harder to read than names.

```css
.box-greet p {
    color: rgb(52, 152, 219);
}
```



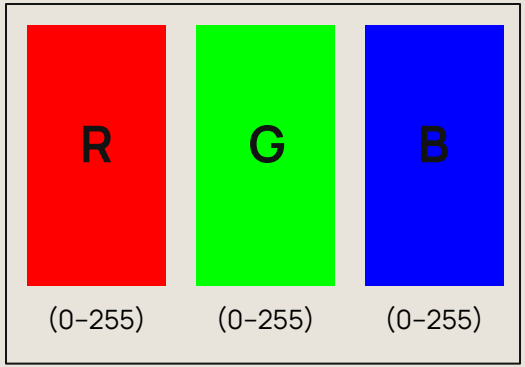


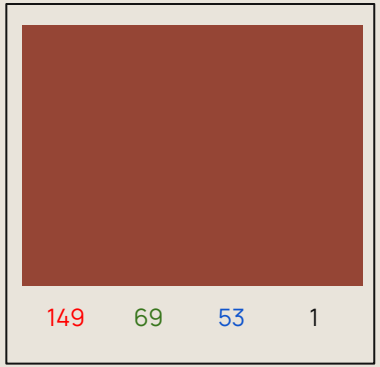


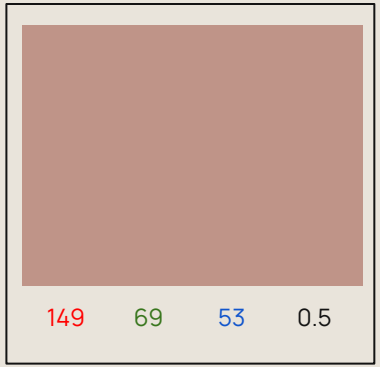
RGBA

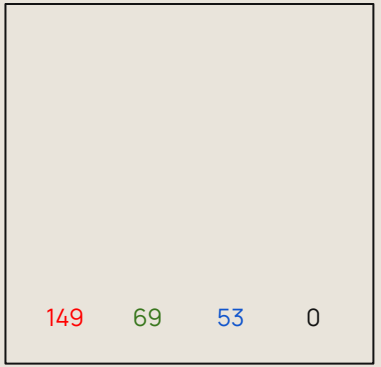
- like RGB but adds an alpha value for transparency (0–1).
- Great for overlays and hover effects.

```css
.box-greet p {
    color: rgba(52, 152, 219, 0.5);
}
```









HSL

- stands for Hue, Saturation, and Lightness.
- It's easier to make lighter or darker versions by changing the Lightness value.

```css
.box-greet p {
    color: hsl(204, 70%, 53%);
}
```

= 204, 70%, 53%

Hue = 204° on color wheel

Saturation = 70%

Lightness = 53%

Bonus: CSS Variables

- In CSS, you can create **variables (custom properties)** to store color values, so you can reuse them and update easily.
- This makes your design more organized and consistent.

```css
:root {
    --primary-color: #3498db;
    --secondary-color: #f39c12;
    --accent-color: #3498db;
    --black-color: #333333;
    --white-color: #efefef;
}
```

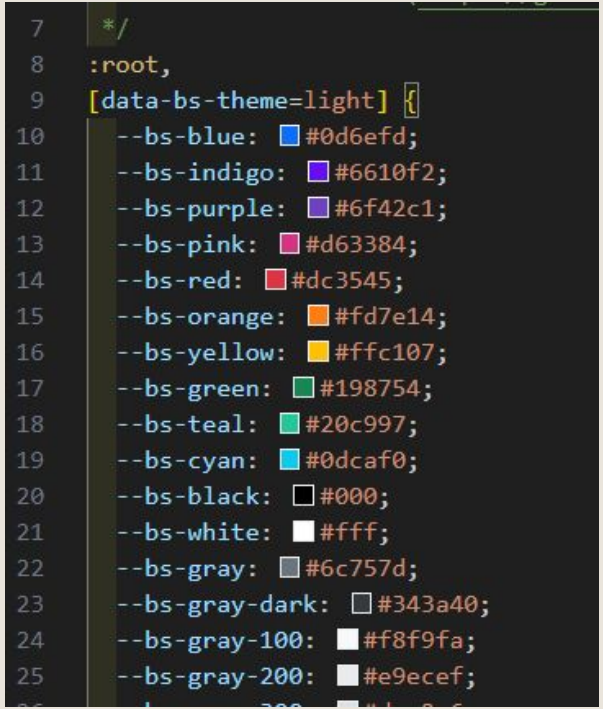
Usage

```css
p{
    color:: var(--primary-color);
    background-color:: var(--white-color);
}
```

CSS Functions

The CSS Variables

- CSS variables (also called CSS custom properties) are like reusable placeholders for values in your CSS, such as colors, font sizes, spacing, etc. They help you write cleaner, more maintainable, and consistent styles across your project.
- CSS variables let you define a value once and use it anywhere in your CSS.



Variable Syntax

- In CSS, you can declare a custom property using two dashes as a prefix for the property name, or by using the @property at-rule. The following sections describe how to use these two methods.

```
:root {
  --primary-color: #2f82ff;
  --secondary-color: #afafaf;
  --cWhite: #ffffff;
  --cBlack: #000000;
  --text-red: #ff4444;
  --text-blue: #5846ff;
}
```

Variable Syntax

Reusing a variable to style 🔥

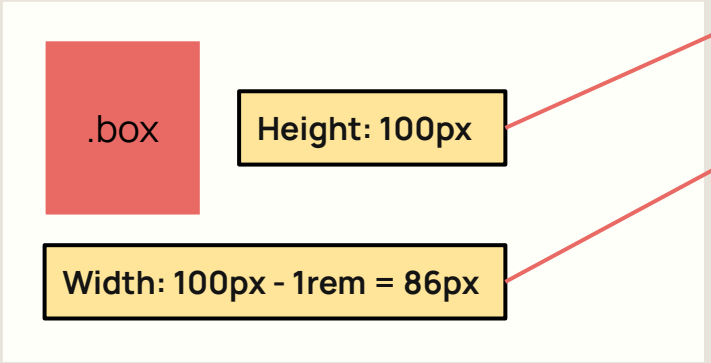
```css
:root {
  --primary-color: #2f82ff;
  --secondary-color: #afafaf;
  --cWhite: #ffffff;
  --cBlack: #000000;
  --text-red: #ff4444;
  --text-blue: #5846ff;
}
```

```css
.button {
  background: var(--primary-color);
  color: var(--cWhite);
  padding: 10px 20px;
  border-radius: 8px;
}
```

The CSS Calculation

- calc() in CSS is a function that lets you do basic math operations directly in your styles.
- allows you to mix units (like % + px) and do math (like +, -, *, /) to make your layout more flexible and dynamic.

```
.box {
  background: red;
  height: 100px;
  width: calc(100px - 1rem);
}
```



Backgrounds

CSS Backgrounds

- **Backgrounds** let you fill an element (like a box, section, or the whole page) with color, images, or patterns.
- They make your designs look more interesting and highlight important areas.

```
.box {
   background: url("bg.jpg") no-repeat center/cover lightblue;
}
```

CSS Backgrounds

Property	Description
background-color	Sets the background color (e.g., #f0f8ff or lightblue)
background-image	Adds an image behind the element (e.g., url('bg.jpg'))
background-image	Decides if the image repeats (repeat, no-repeat, repeat-x, repeat-y)
background-position	Sets where the image starts inside the element (center, top left, 10px 20px, etc.)
background-size	Changes the size of the image (cover, contain, or specific size like 100px 50px)

CSS Backgrounds

Longhand Code

```
.box {
    background-color: #f0f8ff;
    background-image: url("https://picsum.photos/200/300");
    background-repeat: no-repeat;
    background-position: center;
    background-size: cover;
}
```

CSS Backgrounds

Shorthand Code

```
.box {
    background: url("https://picsum.photos/200/300") no-repeat
center/cover #f0f8ff;
}
```

background: [background-color] [background-image] [background-repeat] [background-position]/[background-size];

Units in CSS

Units in CSS

- A **unit** in CSS is used to define the **measurement** of values—such as **lengths, sizes, spacing**, and more.
- It tells the browser *how much* of something you want—like how wide a box should be, how big the text should appear, or how much space to leave between elements.

```
.box {
    height: 40px;
    width: 40px;
}
```

```
.text-content {
    font-size: 1rem;
}
```

Units in CSS

Type	Description
px (pixel)	Pixels static point (most commonly used)
%	Value depends on the parent element
em	Relative to the font size of the parent
rem	Relative to root element font size, usually <html>
vw (view width)	Relative to 1% of viewport width
vh (view height)	Relative to 1% of viewport height

Shadows

CSS Shadows

- **Shadows** in CSS let you add soft or sharp drop shadows behind boxes, text, or other elements.
- They help create depth, make parts of the page stand out, and add a modern, layered look to your design.

```css
.box {
    box-shadow: 2px 2px 5px gray;
}
```

```css
p {
    text-shadow: 1px 1px 3px black;
}
```

CSS Shadows

Type	Description
box-shadow	Adds shadow behind boxes, cards, buttons, images, etc.
text-shadow	Adds shadow behind text to create depth or glow.

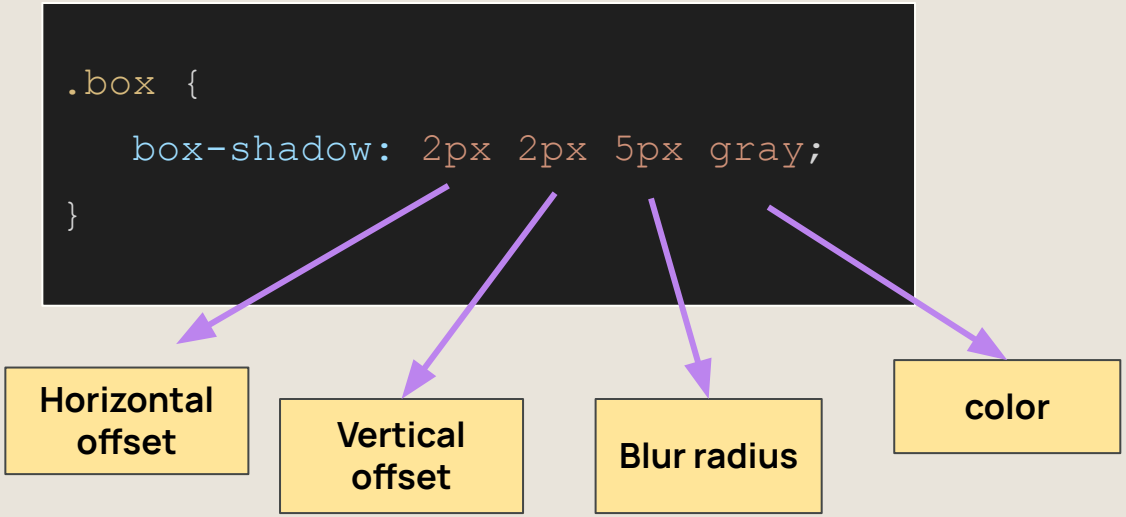
CSS Shadows

Horizontal Offset
- moves shadow right or left

Vertical Offset
- moves shadow up or down

Blur Radius
- makes the shadow softer or sharper

Color
- color of the shadow



- ***Text-shadow*** uses the same pattern
- Negative values for offsets move shadows left or up.

Animations

CSS Animations

- **Animations** in CSS let you make elements move, fade, bounce, rotate, or change style over time — without JavaScript.
- They make websites feel more dynamic and interactive.

CSS Animations

Property	Description
animation-name	Name of the keyframes you created
animation-duration	How long the animation runs (seconds or milliseconds)
animation-timing-function	Controls the speed curve (e.g., ease, linear, ease-in-out)
animation-delay	Waits before starting the animation
animation-iteration-count	How many times to play the animation (1, infinite, 3)
animation-direction	Direction: normal, reverse, alternate (plays back and forth)
animation-fill-mode	Defines what styles apply before/after the animation (none, forwards, backwards, both)

CSS Animations

Create Animation

```
@keyframes slideIn {
  from {
    transform:
translateX(-100px);
  }
  to {
    transform: translateX(0);
  }
}
```

Apply Animation (Longhand Code)

```
.box {
    animation-name: slideIn;
    animation-duration: 2s;
    animation-timing-function :
ease-out;
    animation-delay: 0.5s;
    animation-iteration-count :
infinite;
    animation-direction: alternate;
    animation-fill-mode: forwards;
}
```

CSS Animations

Apply Animation (Shorthand Code)

```
.box {
    animation: slideIn 2s ease-out 0.5s infinite alternate forwards;
}
```

animation: name duration timing-function delay iteration-count direction fill-mode;

Animation Kit

https://angrytools.com/css/animation/

Display

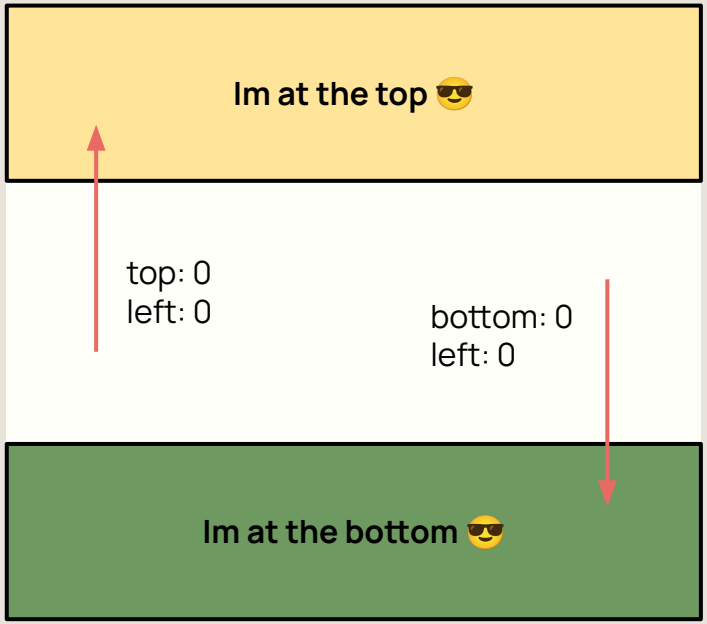
Element	Default Display	Behavior
Block	Starts on new line, takes full width	`<div>`, `<p>`, `<h1>`
Inline	Flows in line, takes only needed width	`<span>`, `<a>`, `<strong>`
inline-block	Flows inline like text but allows width, height, margin, and padding like a block.	`Used for buttons, navigation links, small boxes, icons, etc.`

inline elements that do not accept height and width	
• • <a> • • • <b> • <i> • <u> • <label> • <small> • <abbr>	• <cite> • <time> • <code> • <sub> • <sup> • <q> • <mark> • <dfn> • <kbd>

Positioning

Positioning

- In CSS, positioning refers to how elements are placed on a web page. It determines where an element appears in relation to the normal document flow, the browser window, or other elements.

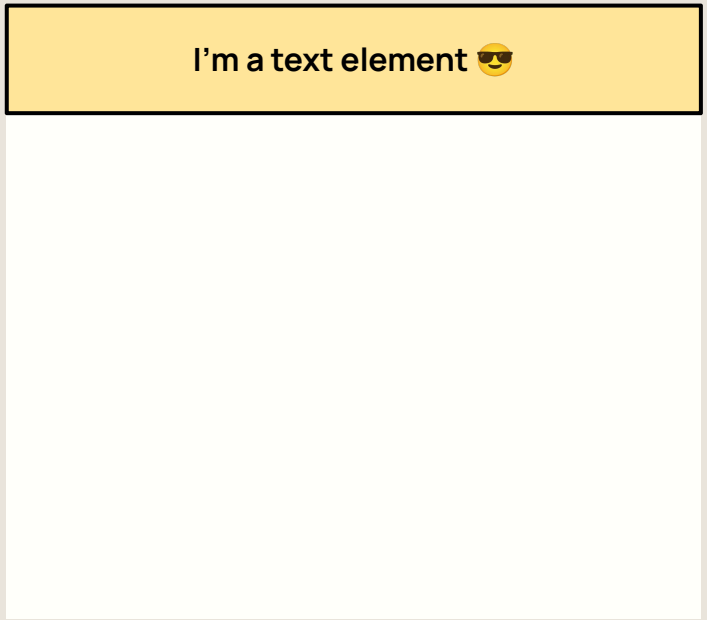


Position overview

Property	Description
static	The element appears in the normal document flow, one after another.
relative	Positions the element relative to its normal position. • It stays in the flow but can be moved using top, left, etc.
absolute	Positions the element relative to the nearest positioned ancestor (not static). • Removed from the normal flow.
fixed	Positioned relative to the browser window. • Stays in the same spot even when you scroll.
sticky	A mix of relative and fixed. • Acts like relative until you scroll to a certain point, then it becomes fixed.

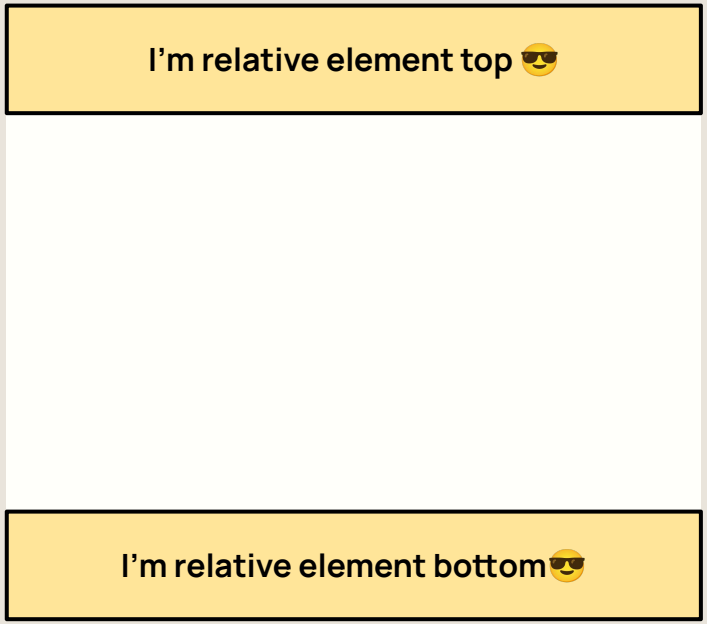
Static

- The element is positioned according to the Normal Flow of the document. The top, right, bottom, left, and z-index properties have no effect. This is the default value.



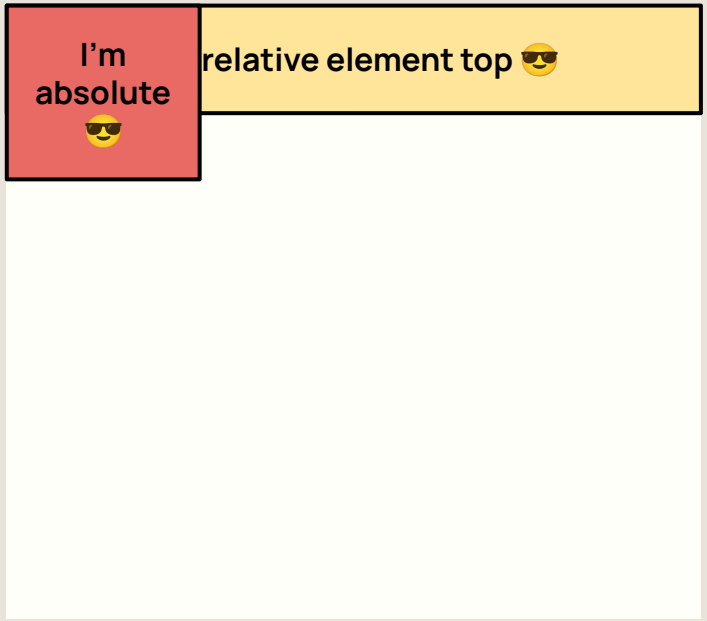
Relative

- The element is positioned according to the normal flow of the document, and then offset relative to itself based on the values of top, right, bottom, and left. The offset does not affect the position of any other elements; thus, the space given for the element in the page layout is the same as if position were static.



Absolute

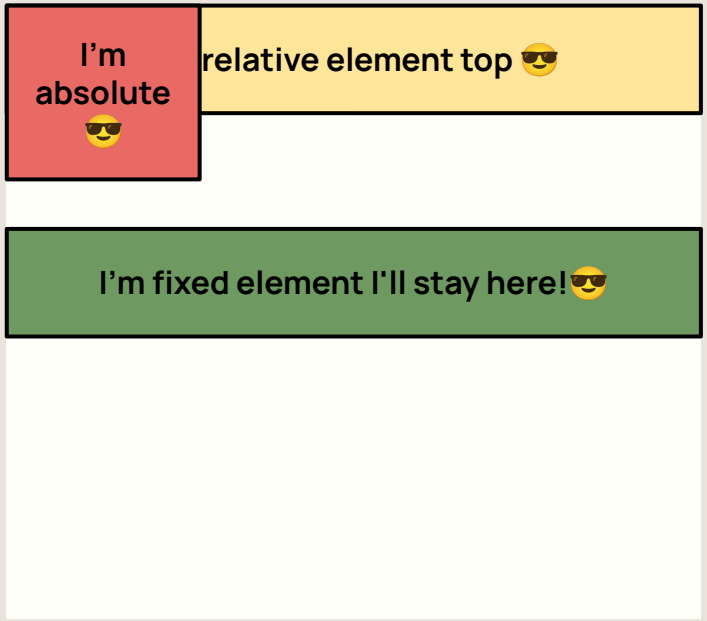
- The element is removed from the normal document flow, and no space is created for the element in the page layout. The element is positioned relative to its closest positioned ancestor (if any) or to the initial containing block. Its final position is determined by the values of top, right, bottom, and left.



Parent is relative

Fixed

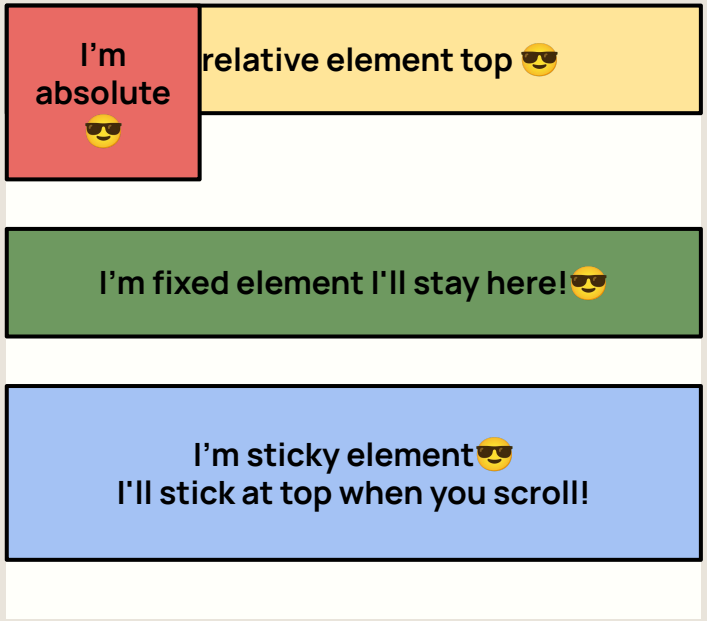
- The element is removed from the normal document flow, and no space is created for the element in the page layout. The element is positioned relative to its initial containing block, which is the viewport in the case of visual media. Its final position is determined by the values of top, right, bottom, and left.



Parent is relative

Sticky

- The element is positioned according to the normal flow of the document, and then offset relative to its nearest scrolling ancestor and containing block (nearest block-level ancestor), including table-related elements, based on the values of top, right, bottom, and left. The offset does not affect the position of any other elements.

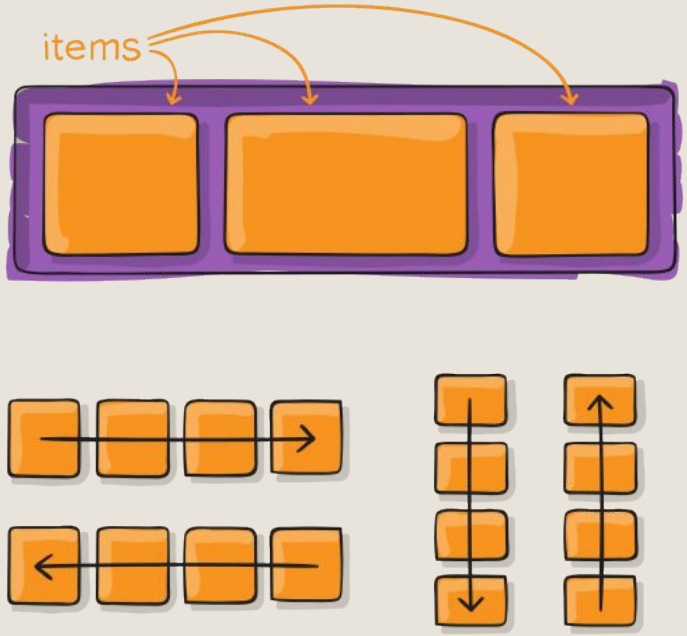


Parent is relative

The Flexbox

What is Flexbox?

- Flexbox (short for Flexible Box Layout) is a CSS layout model that helps you arrange items in a row or column, and control their spacing, alignment, and size even when their size is unknown or dynamic.
- It makes it easier to build responsive layouts without using float or positioning tricks.



Flexbox

`display:` enables flex context for all direct children.

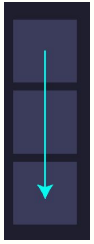
```
.container{
  display: flex // or inline-flex
```

`flex-direction:` sets the main-axis.

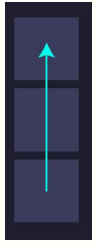
row

row-reverse

column



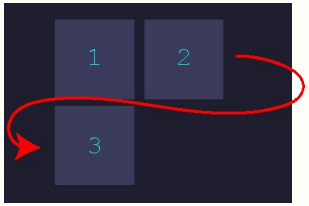
column-reverse



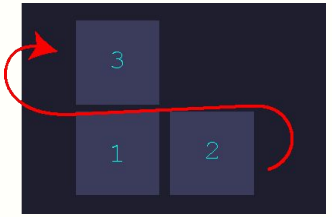
`flex-wrap:` allows the items to wrap as needed.

no wrap

wrap



wrap-reverse



`justify-content:` defines alignment along the main axis.

flex-start

flex-end

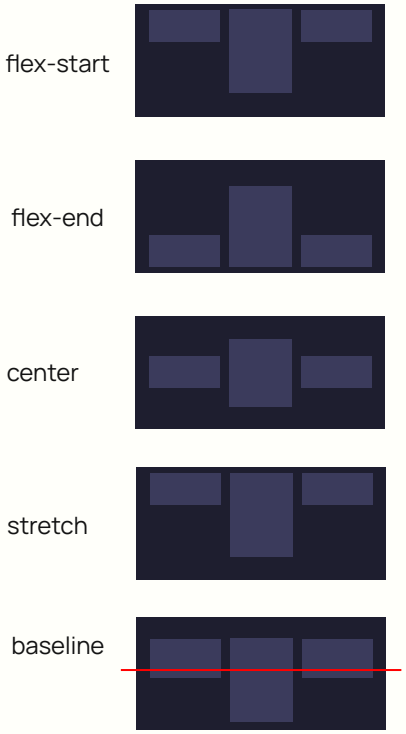
center

space-between

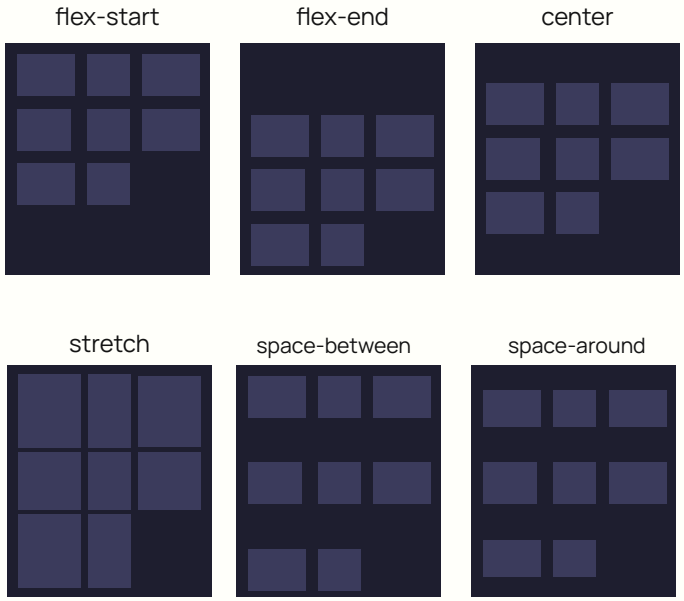
space-around

space-evenly

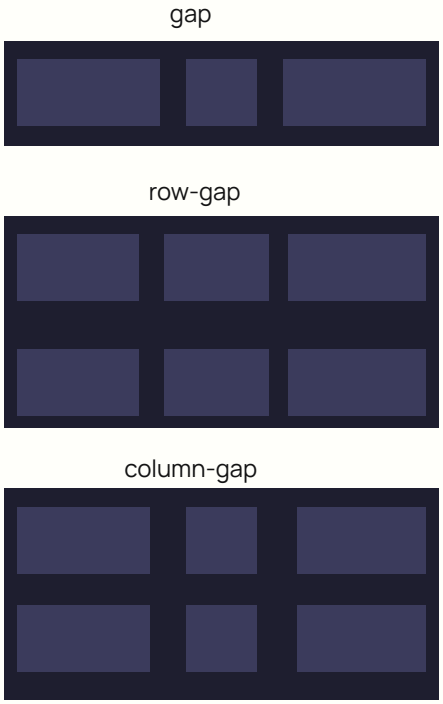
`align-items:` defines alignment along the cross axis.



`align-content:` aligns multiple lines, like justify-content does with individual items.



`gap, row-gap, column-gap:` explicitly controls the space between flex items. It applies that spacing only between items



The Flexbox Overview

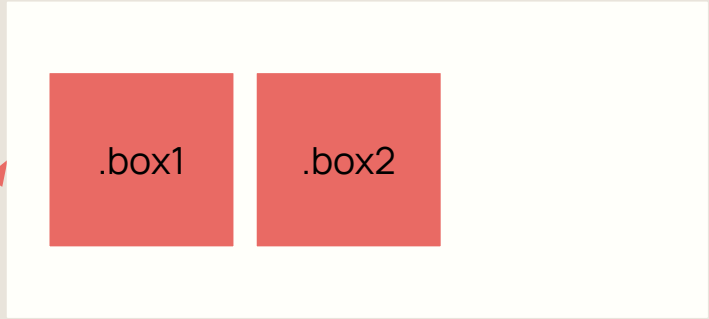
Property	Description
display: flex	Turns an element into a flex container
flex-direction	Sets the direction of items (row, column, etc.)
justify-content	Aligns items horizontally (like center, space-between)
align-items	Aligns items vertically (like center, flex-start)
flex-wrap	Allows items to wrap onto multiple lines if needed
gap	Sets space between flex items (like a margin between them)

The Flexbox syntax

- flex: this defines a flex container; inline or block depending on the given value. It enables a flex context for all its direct children.

```
.container {
    display: flex;
}
```

.container (parent)

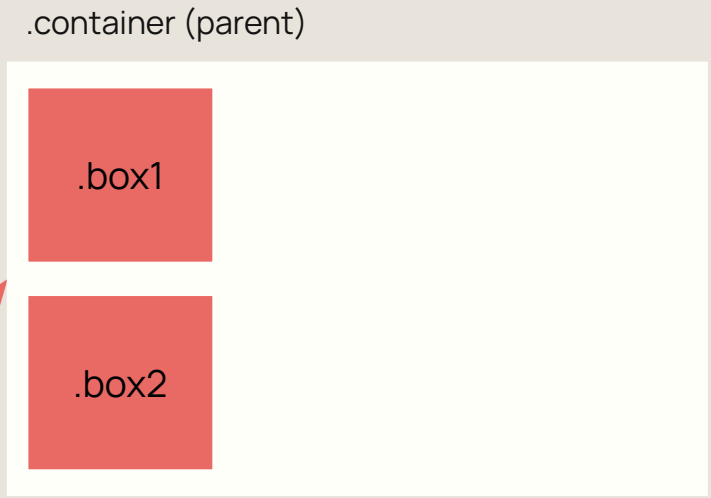


Note: make sure to have a parent element that will hold 2 div or element to use display: flex property 💡

The Flexbox syntax

- flex-direction: This establishes the main-axis, thus defining the direction flex items are placed in the flex container.

```
.container {
    display: flex;
    flex-direction: column;
}
```

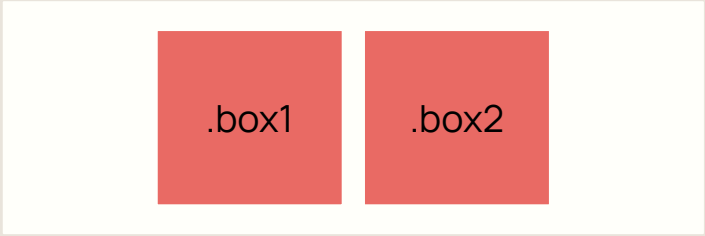


The Flexbox syntax

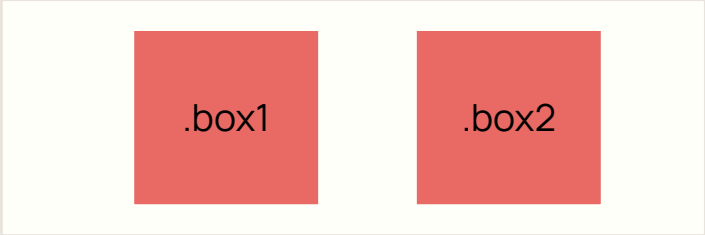
- justify-content: this defines a flex container; inline or block depending on the given value. It enables a flex context for all its direct children.

```
.container {
    display: flex;
    justify-content: center;
}
```

.container (center)



.container (space-around)



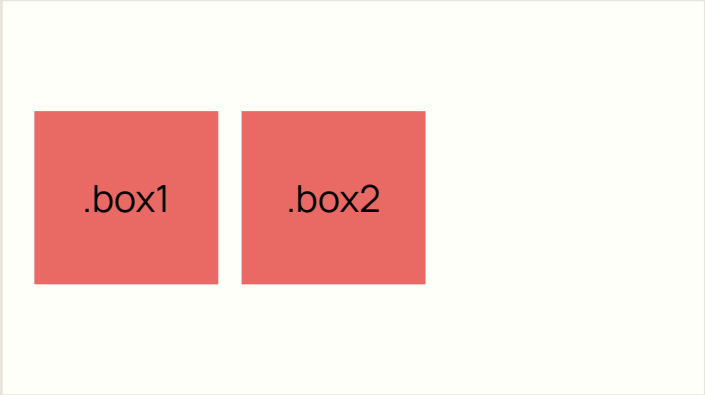
.container (space-between)

The Flexbox syntax

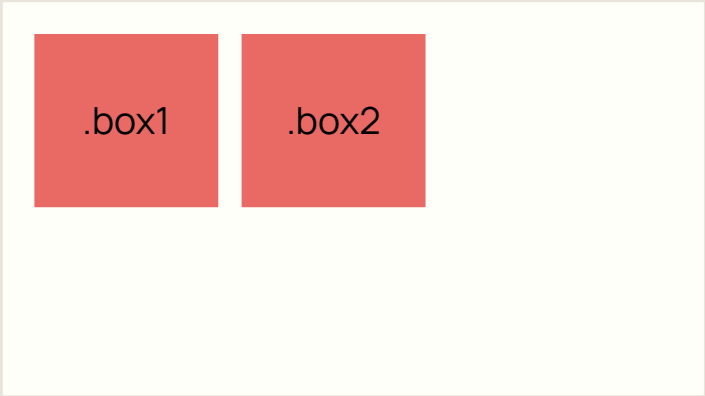
- align-items: This defines the default behavior for how flex items are laid out along the cross axis on the current line. Think of it as the justify-content version for the cross-axis (perpendicular to the main-axis).

```
.container {
    display: flex;
    align-items: center;
}
```

.container (center)

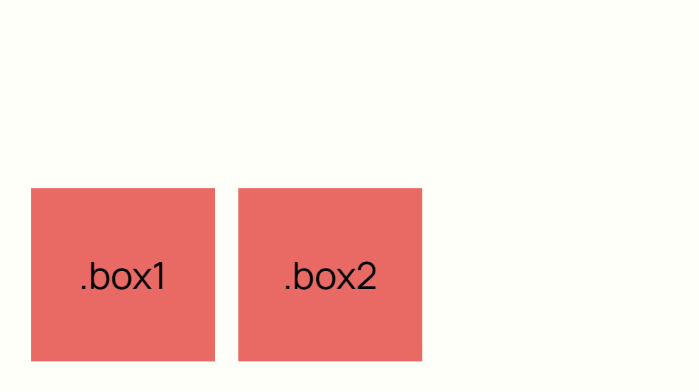


.container (flex-start)

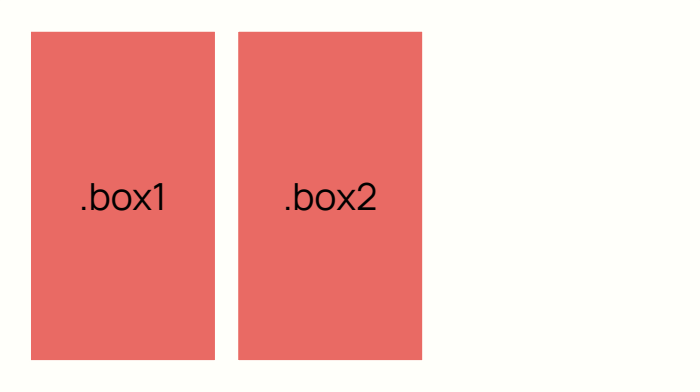


The Flexbox syntax

.container (flex-end)



.container (stretch)

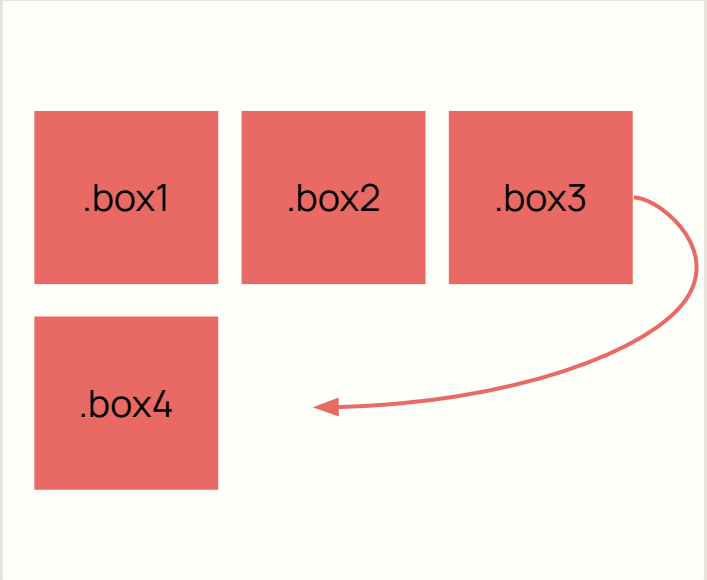


The Flexbox syntax

- flex-wrap: The flex-wrap CSS property sets whether flex items are forced onto one line or can wrap onto multiple lines. If wrapping is allowed, it sets the direction that lines are stacked.

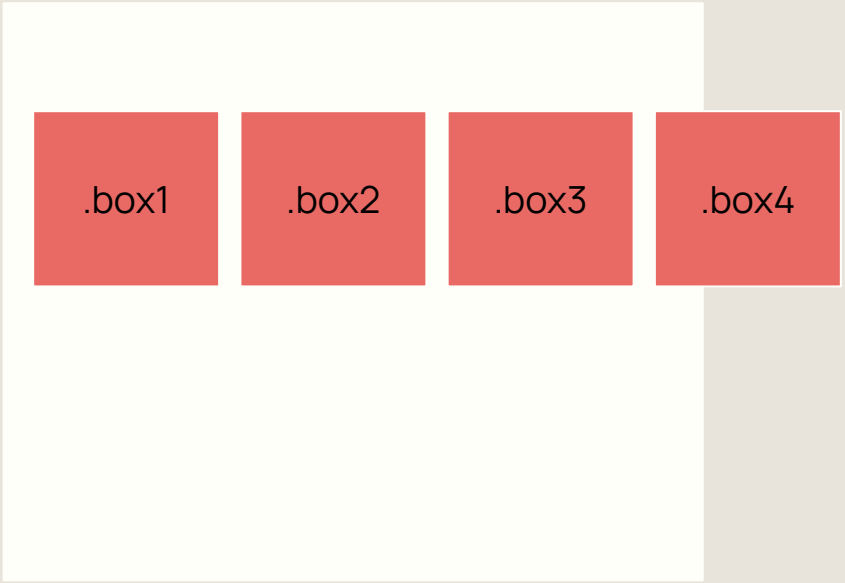
```
.container {
    display: flex;
    flex-wrap: wrap;
}
```

.container (flex-wrap: wrap)

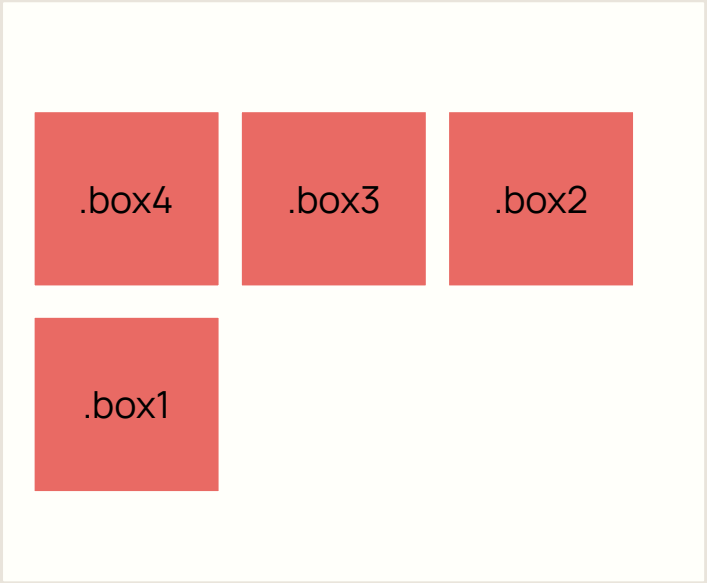


The Flexbox syntax

.container (flex-wrap: nowrap)



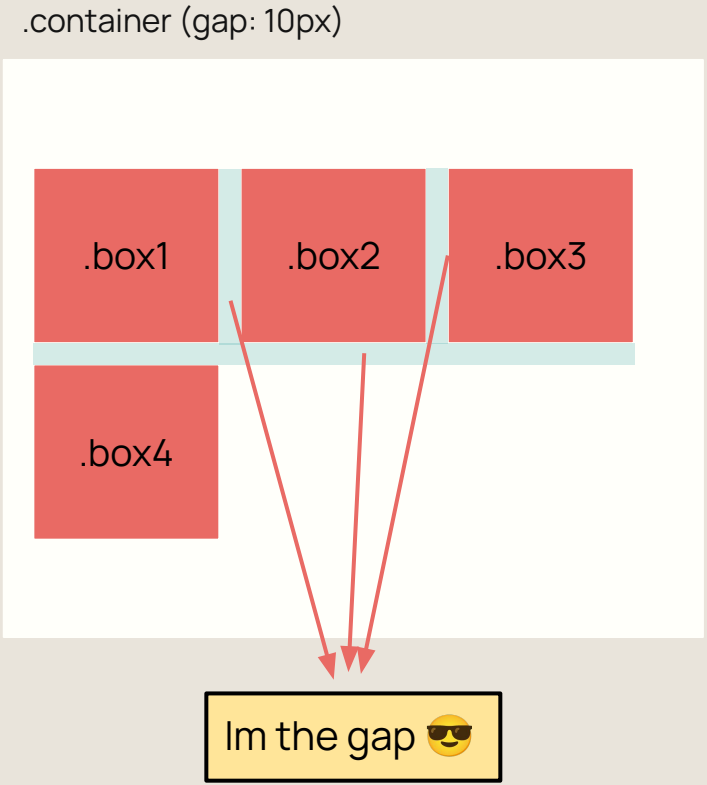
.container (flex-wrap: wrap-reverse)



The Flexbox syntax

- gap: The gap property explicitly controls the space between flex items. It applies that spacing only between items not on the outer edges.

```
.container {
    display: flex;
    gap: 10px;
}
```



The Flexbox syntax

- gap: can be also customized via horizontal and vertical axis

Shorthand code

```
.container {
    display: flex;
    gap: 10px;
}
```

```
.container {
    display: flex;
    gap: 10px 20px;
}
```

Longhand code

```
.container {
    display: flex;
    row-gap: 10px;
    column-gap: 20px;
}
```

Available Resources

Complete documentations

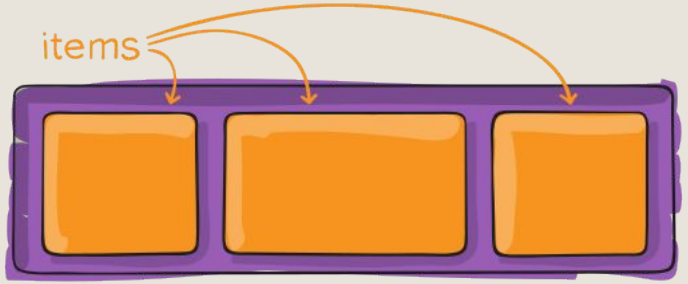
- MDN Web Docs
- CSS Tricks

Generators

- Flexbox Generator

Cheatsheet

- Flex cheatsheet
- Flex visual cheatsheet



The CSS Grid

What is CSS Grid?

- CSS Grid is a layout system in CSS that allows you to design web pages using a grid-based structure made up of rows and columns. It gives you full control over where elements are placed on the page, making it great for creating complex, responsive layouts.

The Grid Overview

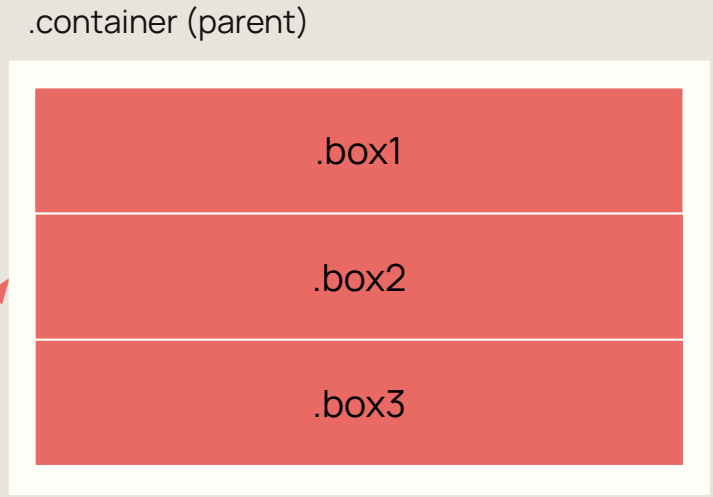
Property	Description
display: grid	Turns an element into a grid container
grid-template-columns	Defines the number and size of columns.
grid-template-rows	Defines the number and size of columns.
grid-column-gap	Sets the space between columns. (Deprecated, use gap instead)
grid-row-gap	Sets the space between rows. (Deprecated, use gap instead)
gap	Shorthand to set both row and column gaps.

Property	Description
grid-template-areas	Defines named grid areas for layout using a text-based pattern
grid-column	Shorthand for setting start and end positions of a grid item in columns.
grid-row	Shorthand for setting start and end positions of a grid item in rows.
grid-area	Assigns a grid item to a named area defined in grid-template-areas.
grid-auto-columns	Sets the size of implicitly-created columns.
grid-auto-rows	Sets the size of implicitly-created rows.
grid-auto-flow	Controls the direction items are placed (row or column) when auto-placing.

The Grid syntax

- grid: Defines the element as a grid container and establishes a new grid formatting context for its contents.

```
.container {
    display: grid;
}
```

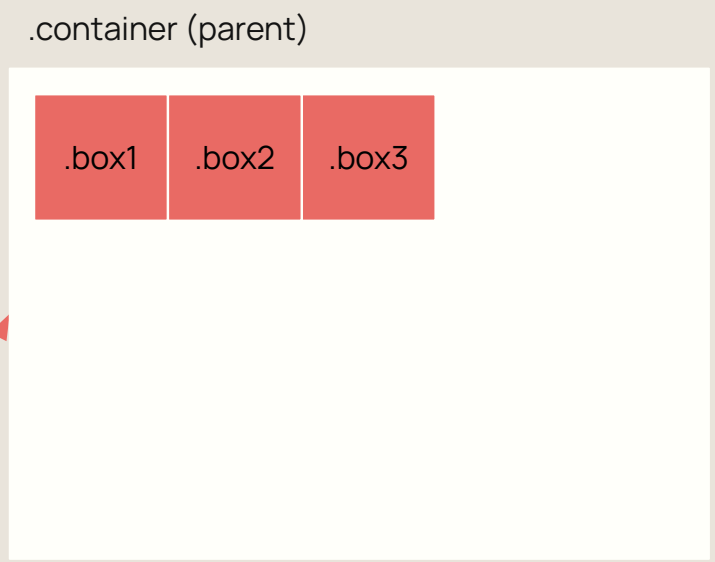


💡 Displaying in grid will not affect anything. Need to add more properties

Grid templates

- grid-template-columns: Defines the line names and track sizing functions of the grid columns.

```
.container {
   display: grid;
   grid-template-columns: 100px 100px 100px;
}
```



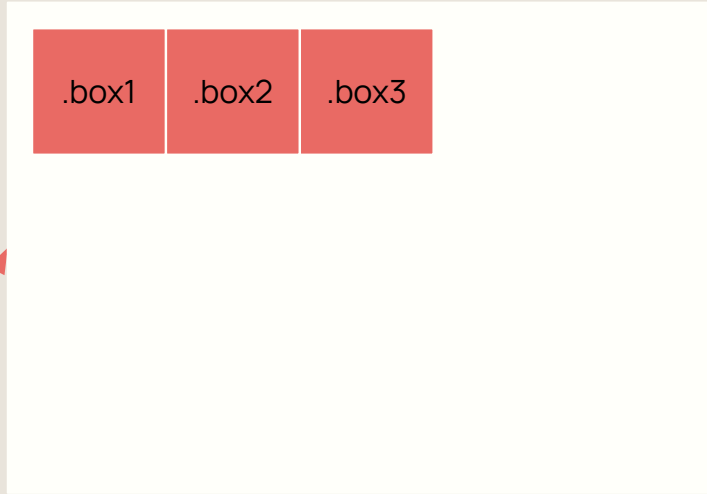
💡 grid-template-columns can accept any form of unit sizing (px, rem, %, vw, fr, calc and etc)

Grid templates (column)

- grid-template-columns: Defines the line names and track sizing functions of the grid columns.

```
.container {
   display: grid;
   grid-template-columns: 100px 100px 100px;
}
```

.container (parent)



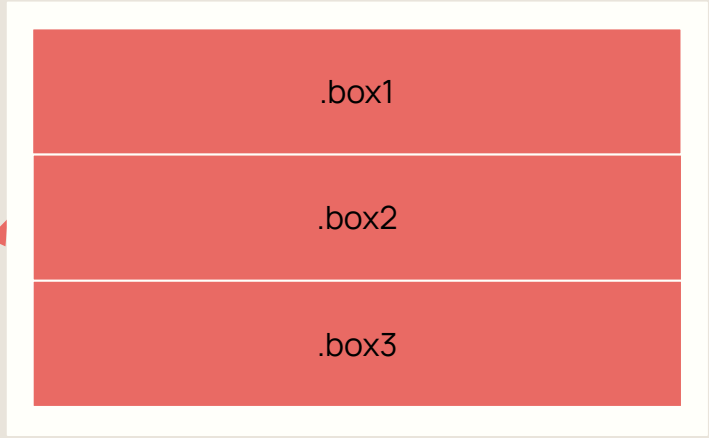
💡 grid-template-columns can accept any form of unit sizing (px, rem, %, vw, fr, calc and etc)

Grid templates (row)

- Grid-template-rows: Property defines the line names and track sizing functions of the grid rows.

```
.container {
   display: grid;
   grid-template-rows: 100px 100px 100px;
}
```

.container (parent)

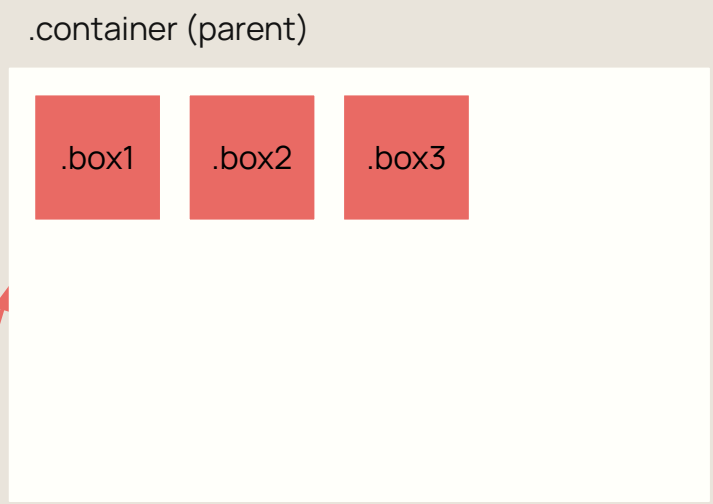


💡 grid-template-rows can accept any form of unit sizing (px, rem, %, vw, fr, calc and etc)

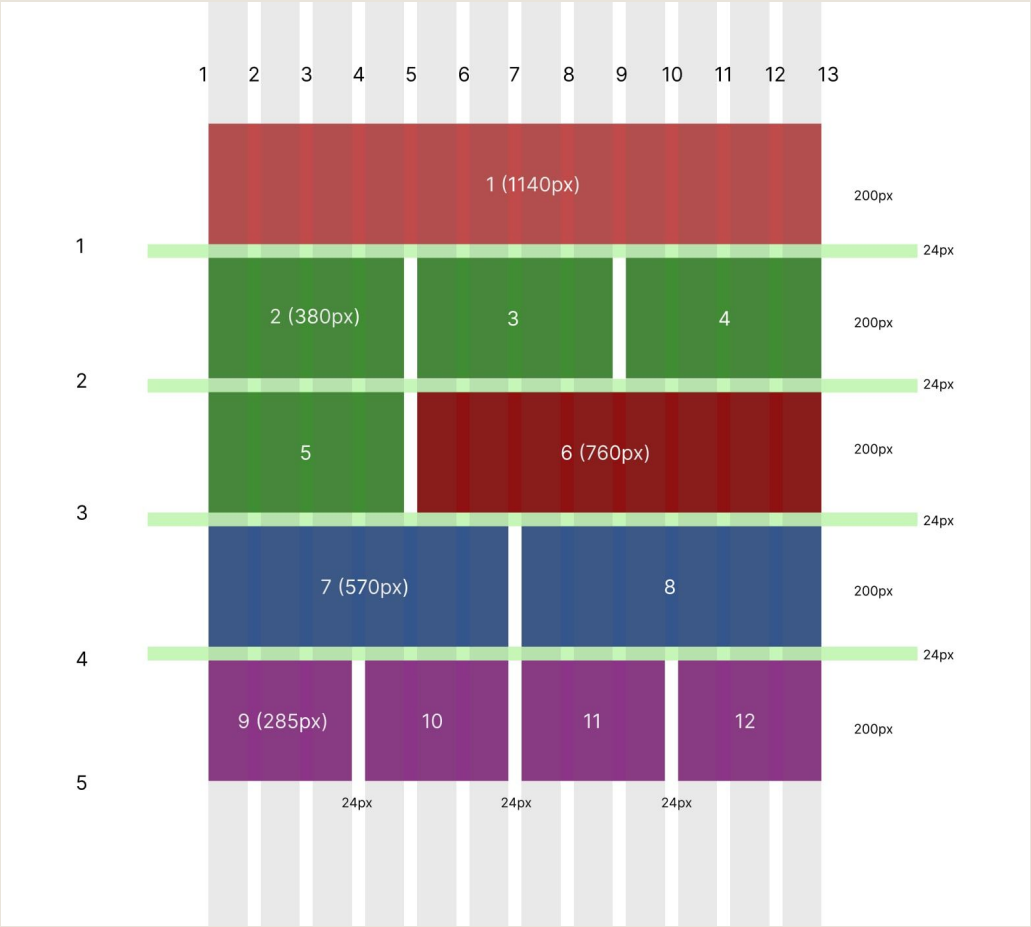
Grid gaps

- gap: defines the size of the gap between the rows and between the columns in flexbox, grid or multi-column layout A shorthand for row-gap and column-gap

```
.container {
   display: grid;
   grid-template-columns: 100px 100px 100px;
   gap: 0 20px;
}
```



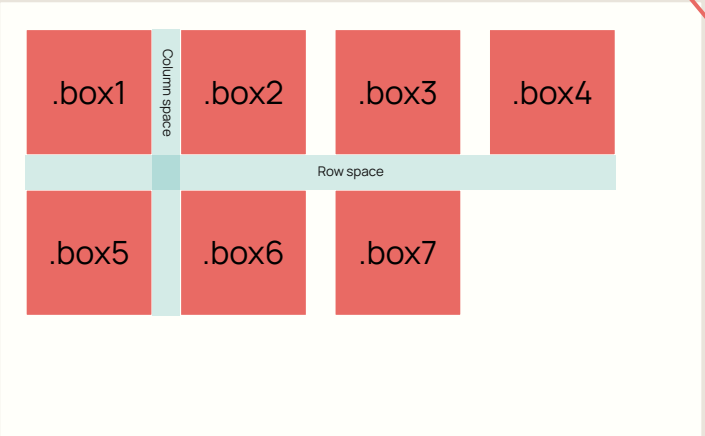
💡 gap can accept any form of unit sizing (px, rem, %, vw, calc and etc) except for fr



Property	Purpose
`grid-column-start`	Defines the **starting vertical grid line** where the item begins (left edge).
`grid-column-end`	Defines the **ending vertical grid line** (right edge).
`grid-row-start`	Defines the **starting horizontal grid line** where the item begins (top edge).
`grid-row-end`	Defines the **ending horizontal grid line** (bottom edge).
`grid-column`	Shorthand for **`grid-column-start / grid-column-end`**.
`grid-row`	Shorthand for **`grid-row-start / grid-row-end`**.

Property	Purpose	Example	Explanation
`grid-column-start`	Defines the **starting vertical grid line** where the item begins (left edge).	`grid-column-start: 1;`	The item starts at the first column line.
`grid-column-end`	Defines the **ending vertical grid line** (right edge).	`grid-column-end: 3;`	The item ends *before* column line 3 (spans 2 columns).
`grid-row-start`	Defines the **starting horizontal grid line** where the item begins (top edge).	`grid-row-start: 1;`	The item starts at the first row line.
`grid-row-end`	Defines the **ending horizontal grid line** (bottom edge).	`grid-row-end: 3;`	The item ends *before* row line 3 (spans 2 rows).
`grid-column`	Shorthand for **`grid-column-start / grid-column-end`**.	`grid-column: 1 / 3;`	Equivalent to starting at column line 1 and ending at 3.
`grid-row`	Shorthand for **`grid-row-start / grid-row-end`**.	`grid-row: 2 / span 2;`	Starts at row line 2 and spans 2 rows downward.

```css
.container {
    display: grid;
    grid-template-columns: 100px 100px 100px;
    gap: 20px 20px;
}
```



Defines the space in column

Defines the space in row

Available Resources

Complete documentations

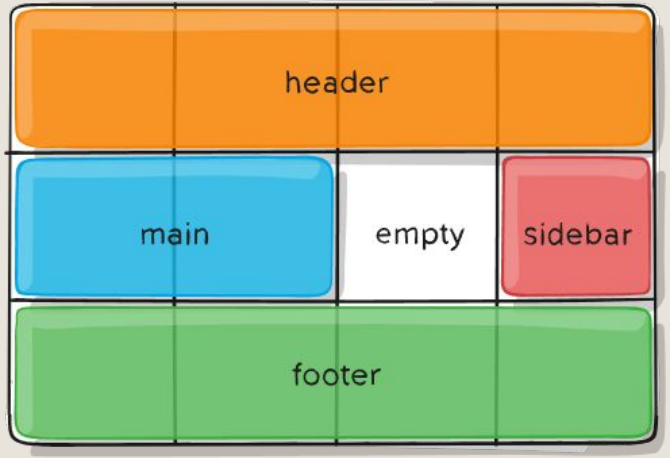
- MDN Web Docs
- CSS Tricks

Generators

- Grid Generator

Cheatsheet

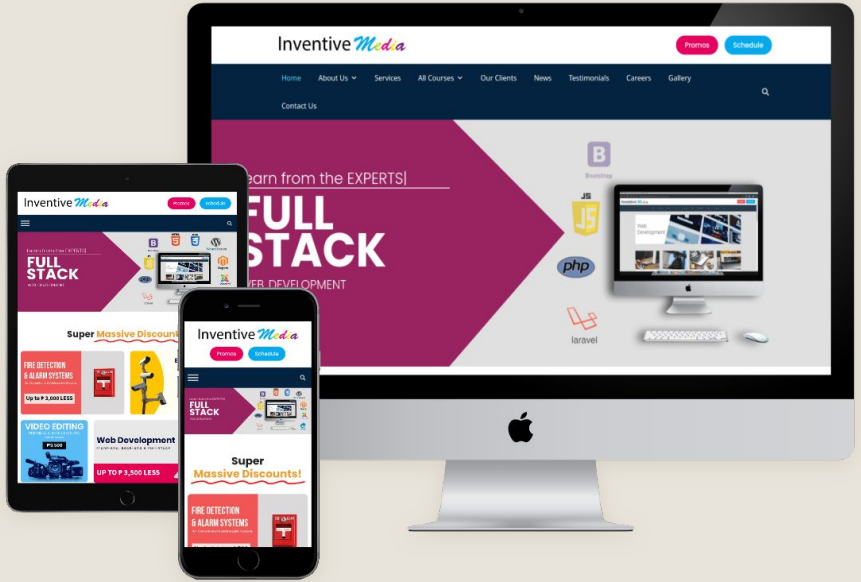
- Grid cheat sheet
- Grid visual cheatsheet



Responsive

What is Responsive?

- Responsive web design (RWD) is a web design approach to make web pages render well on all screen sizes and resolutions while ensuring good usability. It is the way to design for a multi-device web.



Why do we need responsive design?

✅ Looks Good on All Devices

- People use phones, tablets, and computers. A responsive website changes its layout to fit any screen, so it always looks nice and is easy to use.

✅ Helps Your Website Show Up on Google

- Google likes mobile-friendly websites. If your site is responsive, it's easier to find on search engines — which means more people can visit it.

✅ Easier to build and maintain

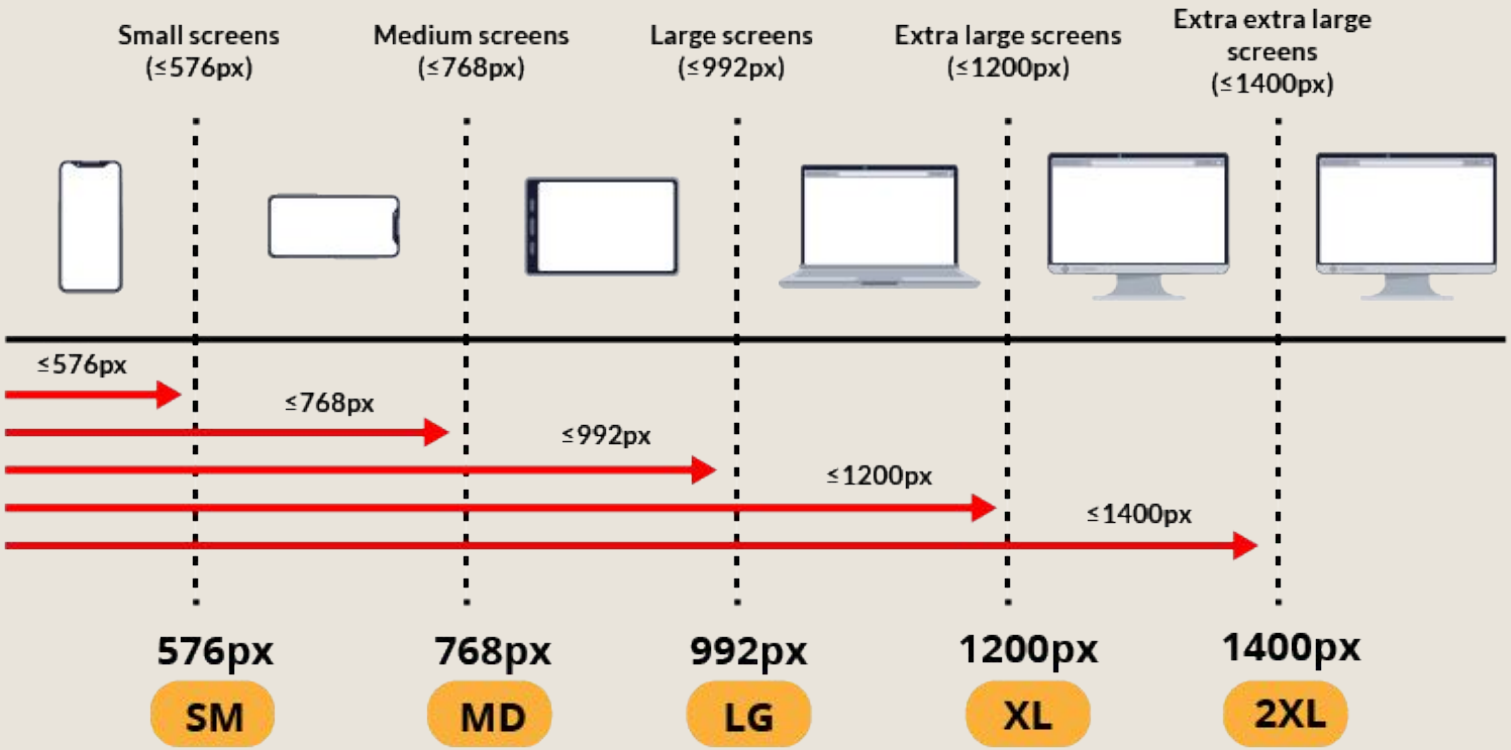
- Instead of creating different versions for phone and desktop, you build one website that works everywhere. This saves time and is easier to update.

Desktop-first approach:

- You start designing for the **largest screens first** (like desktops).
- Then, you use **max-width** media queries to adjust and simplify layouts and features as the screen size decreases.
- It's the opposite of the **"mobile-first"** approach (which uses min-width).

```
/* Extra extra large devices (large laptops and
desktops, 1400px and below) */
@media only screen and (max-width: 1400px) {...}

/* Extra large devices (large laptops and desktops,
1200px and below) */
@media only screen and (max-width: 1200px) {...}

/* Large devices (laptops/desktops, 992px and below) */
@media only screen and (max-width: 992px) {...}

/* Medium devices (landscape tablets, 768px and below)
*/
@media only screen and (max-width: 768px) {...}

/* Small devices (portrait tablets and large phones,
576px and below) */
@media only screen and (max-width: 576px) {...}
```

Desktop-First Approach

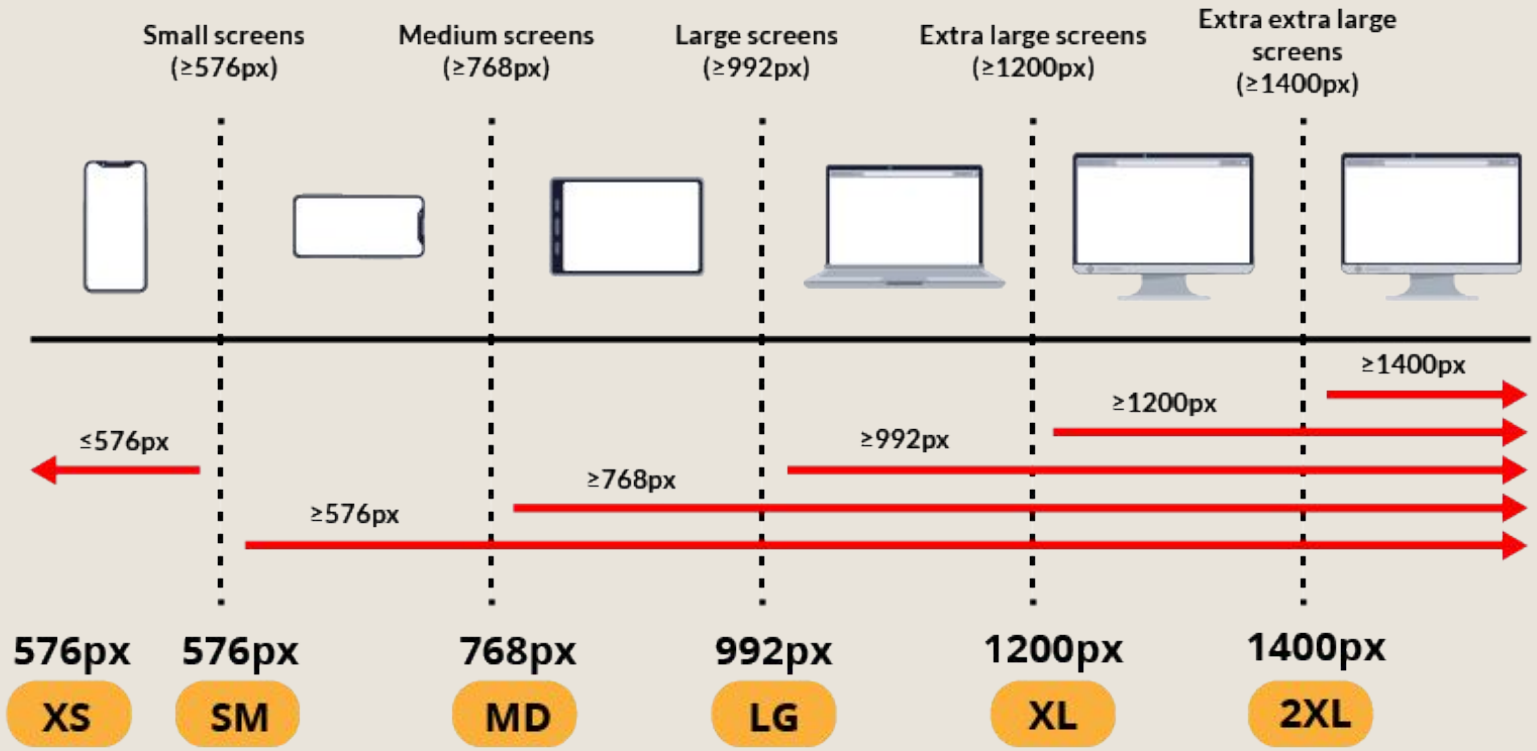


Mobile-first approach:

- You start designing for **smallest screens first** (like smartphones).
- Then, you use **min-width** media queries to gradually **add more complex layouts and features** as the screen size increases.
- It's the **opposite of the "desktop-first"** approach (which uses max-width)

```
/* Extra small devices (phones, 576px and down) */
@media only screen and (max-width: 576px) {...}

/* Small devices (portrait tablets and large phones,
576px and up) */
@media only screen and (min-width: 576px) {...}

/* Medium devices (landscape tablets, 768px and up) */
@media only screen and (min-width: 768px) {...}

/* Large devices (laptops/desktops, 992px and up) */
@media only screen and (min-width: 992px) {...}

/* Extra large devices (large laptops and desktops,
1200px and up) */
@media only screen and (min-width: 1200px) {...}

/* Extra large devices (large laptops and desktops,
1400px and up) */
@media only screen and (min-width: 1400px) {...}
```

Mobile-First Approach



Media Query Width Conditions

Min-Width	Applies *at* or *above* a certain width
Max-Width	Applies *at* or *below* a certain width
Min & Max (Range)	Applies *within* a specific range (min to max)

How RWD works?

```css
h1 {
  color: red;
  background: cadetblue;
}

@media screen and (max-width: 500px) {
  h1 {
    color: blue;
    background: cyan;
  }
}
```

From 500px - above

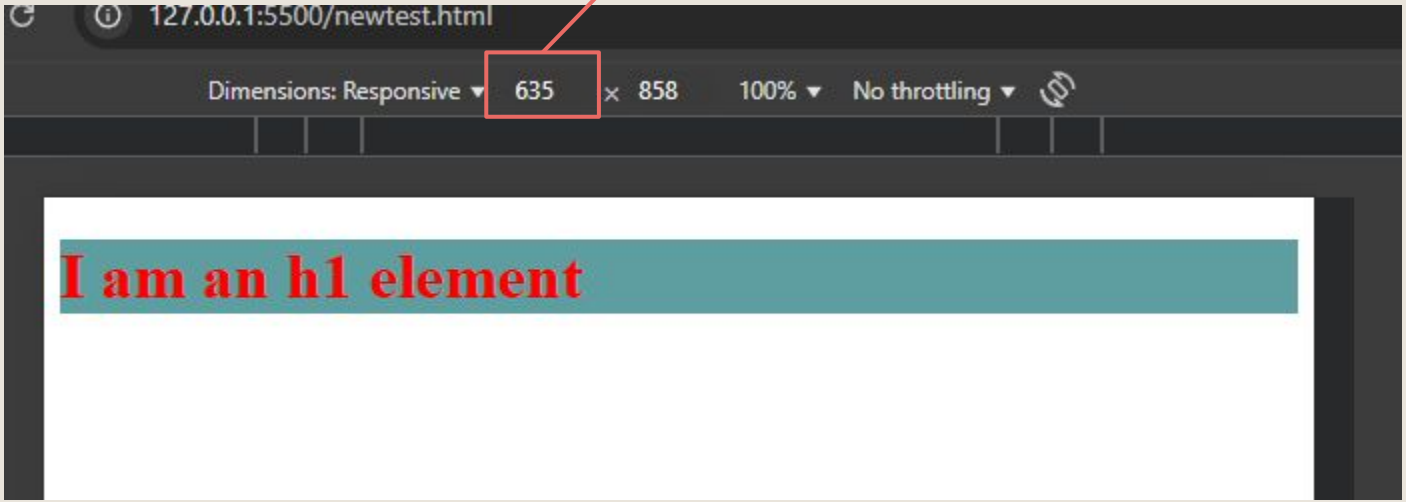
This is the regular styles of h1 as set here on the css

From 0 to 500px the new styles from h1 will be applied here

From browsers POV

From 500px - above, the applied styles are

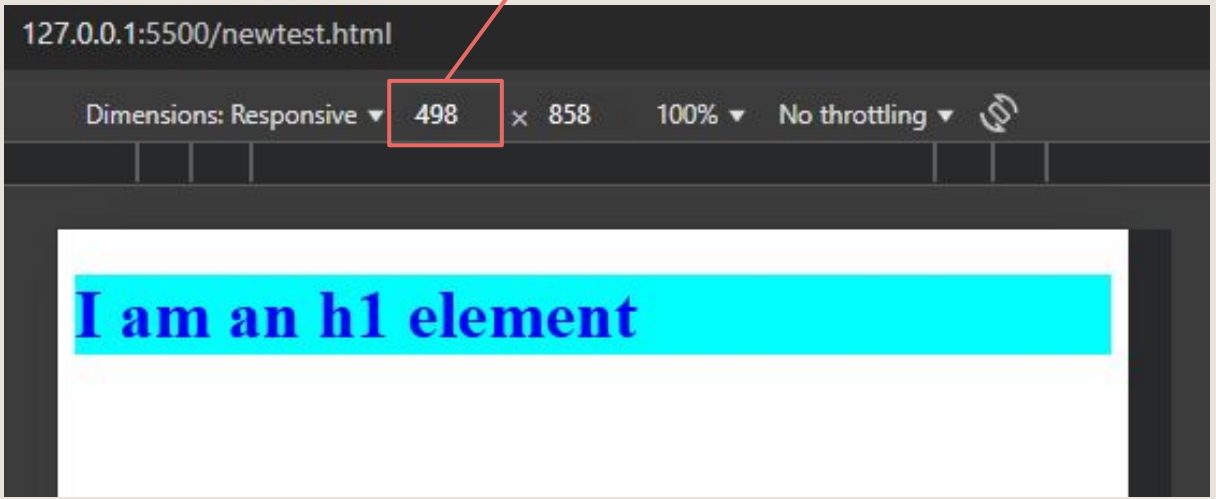
```
color: red;
background: cadetblue;
```



From browsers POV

From 500px - below, the applied styles are

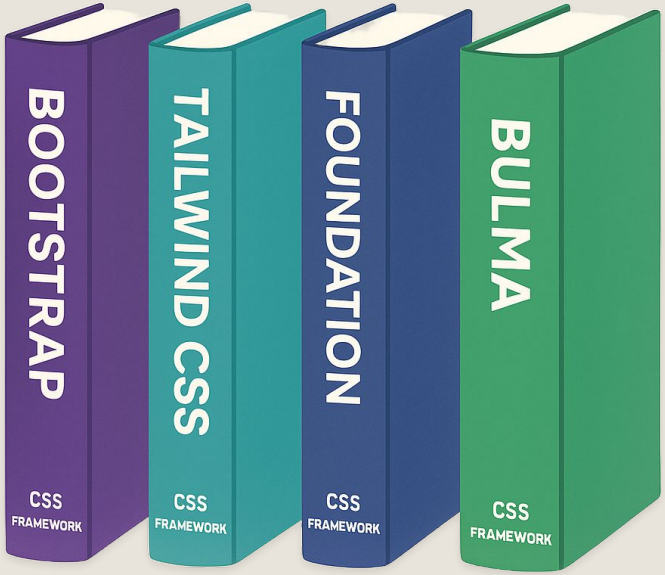
```
color: blue;
background: cyan;
```



CSS Frameworks

What is CSS Framework?

- A CSS framework is a set of ready-made CSS (Cascading Style Sheets) code that helps you style websites faster and easier. Instead of writing all the CSS from scratch, you can use a framework that provides predefined styles for things like buttons, layouts, colors, and typography.



Why use CSS Framework?

- ✅ Saves time – no need to build everything from zero
- ✅ Consistent design – keeps your UI looking clean and uniform
- ✅ Responsive design – most frameworks are mobile-friendly by default
- ✅ Cross-browser support – works well on different web browsers

Without Framework

```
<button class="button">
    I am a button!
</button>
```

```
.button {
    background-color: blue;
    color: white;
    padding: 10px;
}
```

VS

With Framework

```
<button class="btn btn-primary">
    I am a button!
</button>
```

Bootstrap installed 😎

CSS Frameworks

- There are a lot of CSS frameworks available in the internet but these are some of the famous CSS frameworks used today.

Bootstrap CSS
CSS Framework

Tailwind CSS
CSS Framework

Bulma CSS
CSS Framework

Materialize CSS
CSS Framework

Inventive Media 👋

Thank You

Introduction to CSS

Day 2